Distributed coordination

Distributed Systems

Alberto Lafuente, Mikel Larrea

UPV/EHU

Contents

- Introduction
- Distributed mutual exclusion
- Election algorithms
- Group communication
- Replication
- Distributed transactions
- The Consensus problem

Introduction

- In many distributed applications there is a need for coordination/synchronization. Examples:
  - exclusive access to shared resources (mutual exclusion)
  - election of a coordinating/leader process
  - diffusion of messages to a group of processes, with precise reliability and order criteria
  - replication, for fault tolerance or high availability, with precise consistency criteria
  - distributed transactions: ACID properties
- The Consensus problem as a common denominator

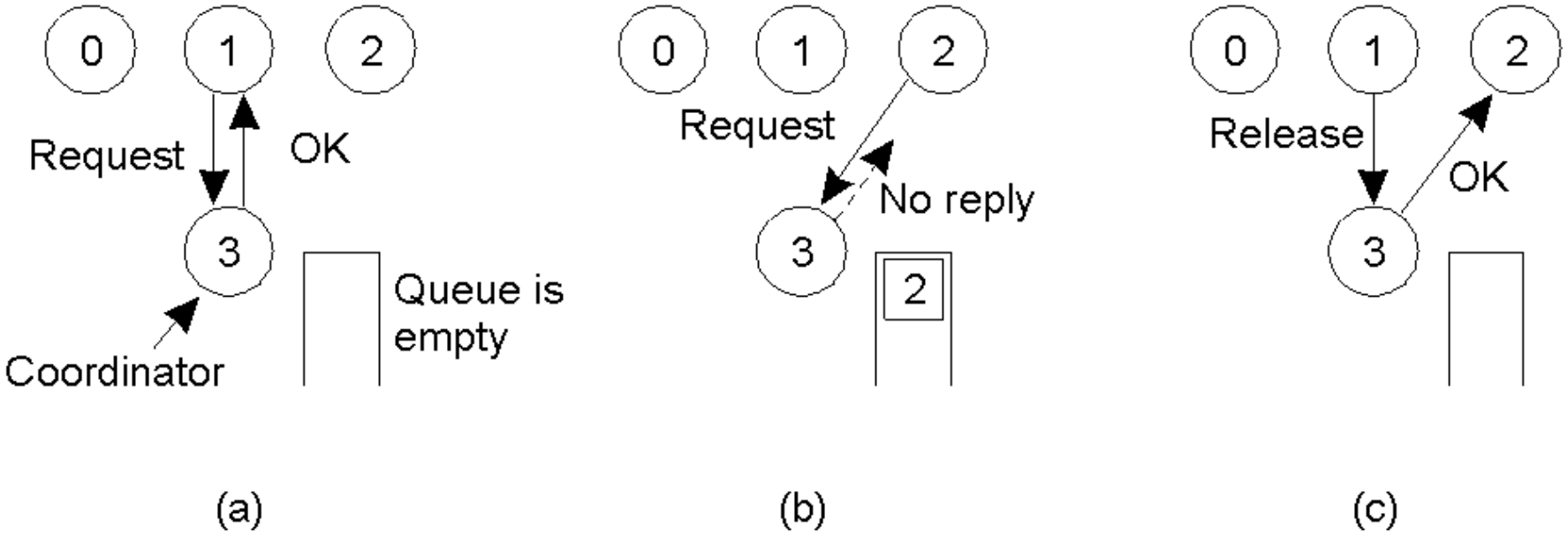
Distributed mutual exclusion

- System model:
  - a set of processes shares a distributed resource
  - a single process can use the resource at a time
- Problem specification:
  - i) if a process is using the resource, it must release it before giving it to another process
  - ii) requests to use the resource must be served by respecting the (causal) order in which they are made
  - iii) if all the processes that get the resource release it after using it, then sooner or later all the requests will be served

Distributed mutual exclusion

- In centralized algorithms, the critical section is managed by a coordinating process
  - request to enter the critical section:
    - send a message to the coordinator and wait for a confirmation
    - if the resource is free, the coordinator replies with a confirmation message (confirmation = authorization)
    - if it is occupied, the coordinator does not reply (or replies with a denial message) and enqueues the request
  - request to leave the critical section:
    - send a message to the coordinator indicating the leave
    - if the queue of pending processes is not empty, the coordinator chooses the first to enter and sends a confirmation message, removing it from the queue

Distributed Mutual Exclusion: A Centralized Algorithm



a) Process 1 asks the coordinator for permission to enter a critical region. Permission is granted

b) Process 2 then asks permission to enter the same critical region. The coordinator does not reply

c) When process 1 exits the critical region, it tells the coordinator, which then replies to 2

Distributed mutual exclusion

- Analysis of centralized algorithms:
  - low scalability: in large systems the coordinator becomes a bottleneck
  - low fault-tolerance: the coordinator is a *single point of failure*. If it fails processes cannot use the resource
    - without denial messages, a failed coordinator is difficult to distinguish from a coordinator that does not reply
  - request order (FIFO): causal order is not guaranteed
- In distributed algorithms there is no coordinator:
  - Algorithm of Ricart and Agrawala
  - Ring algorithm (*token ring*)

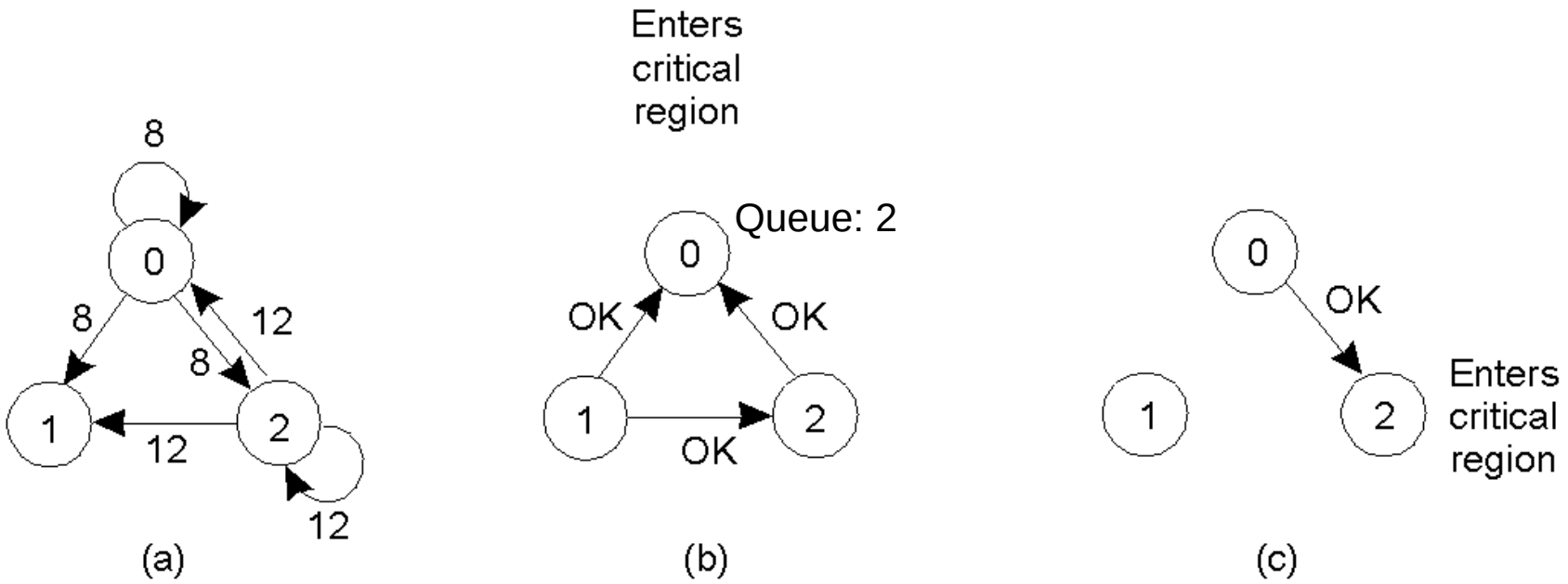
Distributed mutual exclusion

- Algorithm of Ricart and Agrawala:
  - requires a total order in the events
  - request to enter the critical section:
    - send a message to all processes: process identifier + timestamp
    - a receiver process:
      - if it is not in the critical section, it sends a confirmation message
      - if it is in the critical section, it does not reply (or replies with a denial message) and enqueues the request
      - if it is waiting to enter the critical section, it compares the request's timestamp with that of its own request. If the request's timestamp is older, it sends a confirmation message. If not, it does not reply (or denies) and enqueues the request
    - to enter the critical section, a process has to receive a confirmation message from the rest of processes

Distributed mutual exclusion

- Algorithm of Ricart and Agrawala:
  - request to leave the critical section:
    - send a confirmation message to all the processes in the queue, removing them from the queue
- Analysis:
  - *2(n-1)* messages (*n* if multicast support is available)
  - any process failure blocks the system
    - if explicit denial messages are sent, timeouts can be used
  - if the set of processes is dynamic, process joins, leaves and failures must be managed
    - group communication support can be helpful

Distributed Mutual Exclusion: A Distributed Algorithm



a) Two processes want to enter the same critical region at the same moment

b) Process 0 has the lowest timestamp, so it wins

c) When process 0 is done, it sends an OK also, so 2 can now enter the critical region

Ricart and Agrawala's algorithm

On initialization
    state := RELEASED;
To enter the section
    state := WANTED;
    Multicast *request* to all processes;
    T := request's timestamp;
    Wait until (number of replies received = (*N* – 1));
    state := HELD;

request processing deferred here

On receipt of a request $<T_i, p_i>$ *at* p_j $(i \neq j)$
    if (*state* = HELD or (*state* = WANTED *and* $(T, p_j) < (T_i, p_i)$))
    then
        queue *request* from p_i without replying;
    else
        reply immediately to p_i;
    end if
To exit the critical section
    state := RELEASED;
    reply to any queued requests;

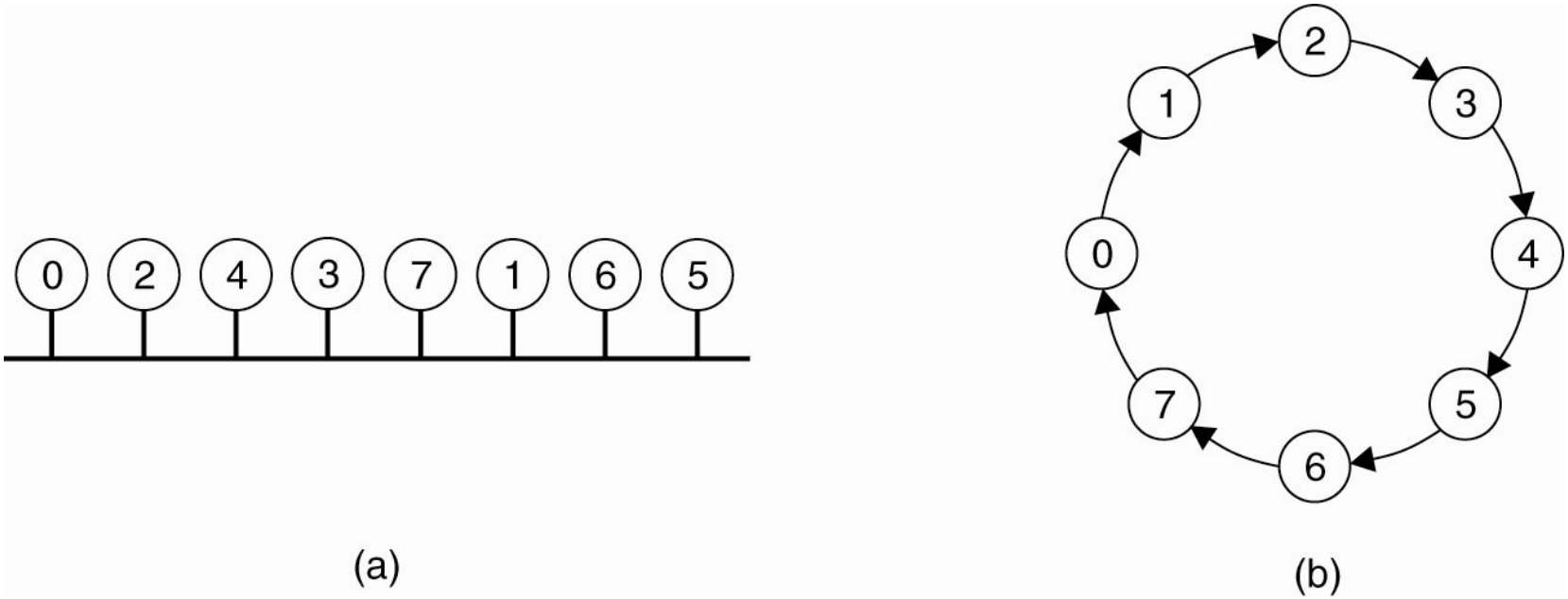
Distributed mutual exclusion

- From Tanenbaum's book:
  - *"Ricart-Agrawala's mutual exclusion algorithm is slower, more complicated, more expensive, and less robust than the original centralized one. Why bother studying it under these conditions? For one thing, it shows that a distributed algorithm is at least possible, something that was not obvious when we started. Also, by pointing out the shortcomings, we may stimulate future theoreticians to try to produce algorithms that are actually useful. Finally, like eating spinach and learning Latin in high school, some things are said to be good for you in some abstract way."*

Distributed mutual exclusion

- Ring algorithm (*token ring*):
  - a logical ring among processes is constructed, and a token message circulates one way through the ring
    - initially, the token is given to a predefined process
    - a process out of the critical section, upon reception of the token, sends it to its successor in the ring
    - when a process want to enter the critical section, it waits to receive the token. When it receives it, it keeps the token and enters the critical section
    - when a process leaves the critical section, it sends the token to its successor in the ring
  - issues: lost token, process failures, causality

Distributed Mutual Exclusion: A Token Ring Algorithm



a) An unordered group of processes on a network

b) A logical ring constructed in software

Distributed Mutual Exclusion Algorithms: Comparison

Algorithm	Messages per entry/exit	Delay before entry (in message times)	Problems
Centralized	3	2	Coordinator crash
Distributed	2(n–1)	2 to 2(n–1)	Crash of any process
Token ring	1 to ∞	0 to n–1	Lost token, process crash

A comparison of three mutual exclusion algorithms

Election algorithms

- Many distributed services are based on a coordinating/leader process. Examples:
  - mutual exclusion (centralized algorithm)
  - distributed transactions (*Two Phase Commit*)
  - replication (*primary-backup* model)
  - In these services, the failure of the coordinator must be detected, in order to elect a new one
- The elected process must be unique
- Assumption: processes are totally ordered
- Examples: bully algorithm, ring algorithms

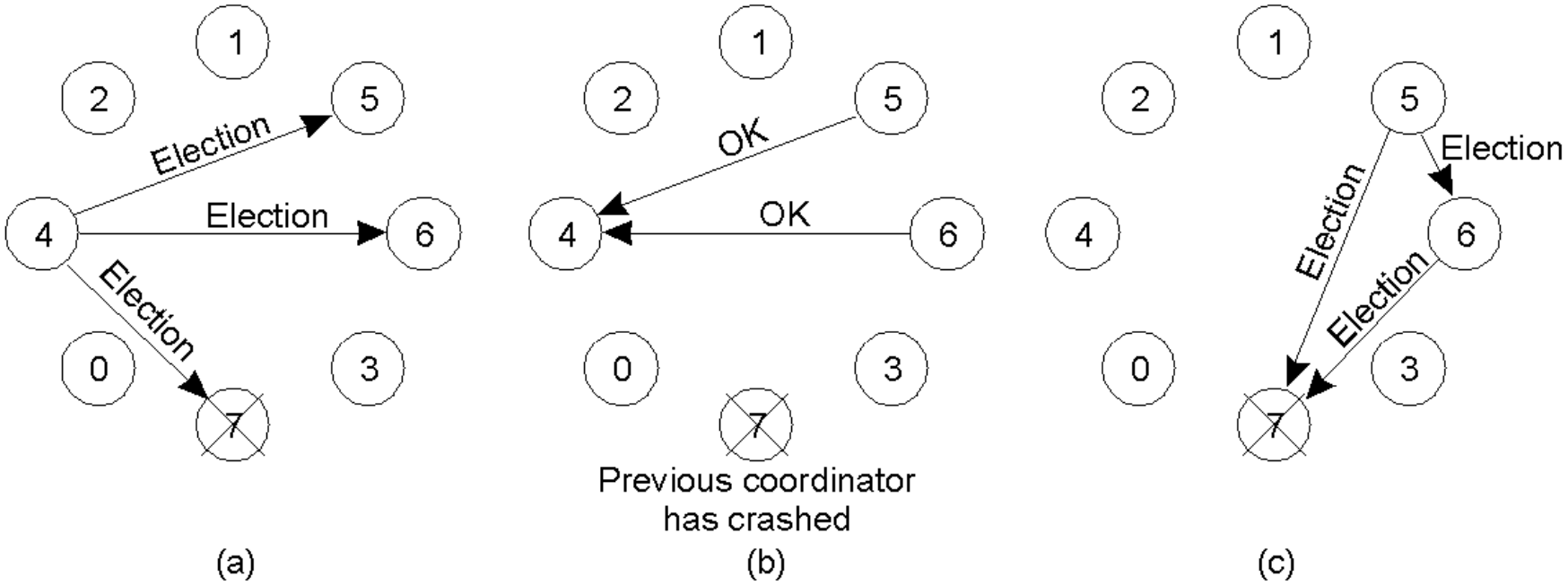
Election algorithms

- System model and properties:
  - any process can initiate the election algorithm
    - a process can initiate one election at a time. In principle, the N processes can initiate N concurrent elections
  - processes are participant or non-participant (yet) in an election
  - each process p_i has a $leader_i$ variable. When a process becomes participant in an election, it sets $leader_i = \perp$, a value which indicates that p_i has not elected a leader yet
  - i) each participant will have $leader_i = \perp$ or $leader_i = P$, being P the process with the highest identifier among those that have not failed
  - ii) all the correct processes participate in the election, setting sooner or later $leader_i \neq \perp$, or failing

Election algorithms

- The bully algorithm (García-Molina):
  - the process *p* that initiates the election sends a *challenge* message to all processes with a higher identifier, becoming eligible
  - if nobody replies, *p* wins the election and becomes the new coordinator, sending a *leader* message to all processes. Otherwise, i.e., if *p* receives a *reply* message (from a process with a higher identifier) then *p* will no longer be eligible
  - when a process *p* receives an election message (from a process *q* with a lower identifier), *p* sends a *reply* message to *q*, and becomes eligible, repeating the procedure (i.e., sending a challenge message to all processes with a higher identifier)
  - in the end there will be a single eligible process, which will be the new coordinator and will inform the rest of processes

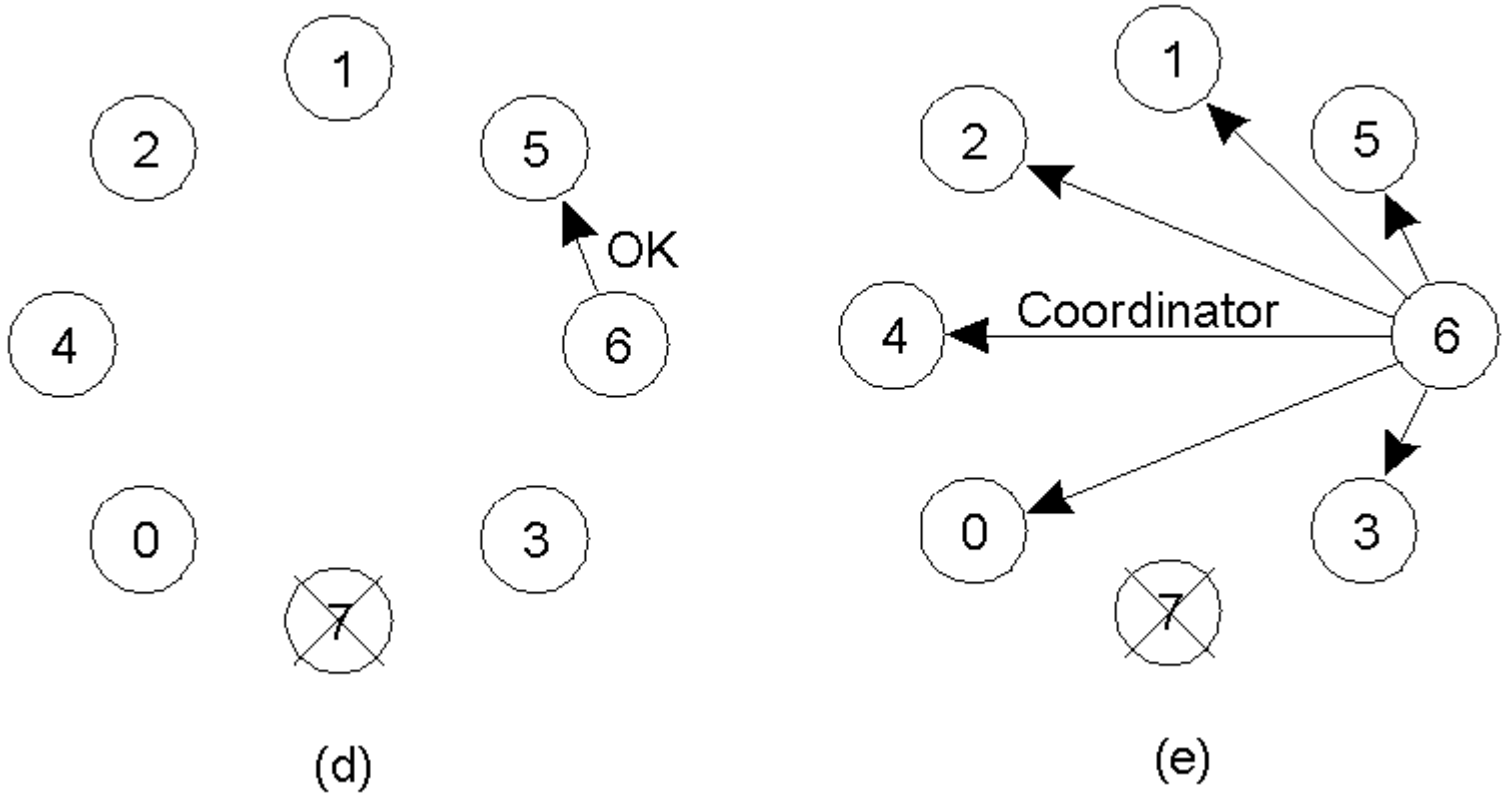
The Bully Algorithm



The bully election algorithm

- Process 4 holds an election
- Process 5 and 6 respond, telling 4 to stop
- Now 5 and 6 each hold an election

The Bully Algorithm (cont.)



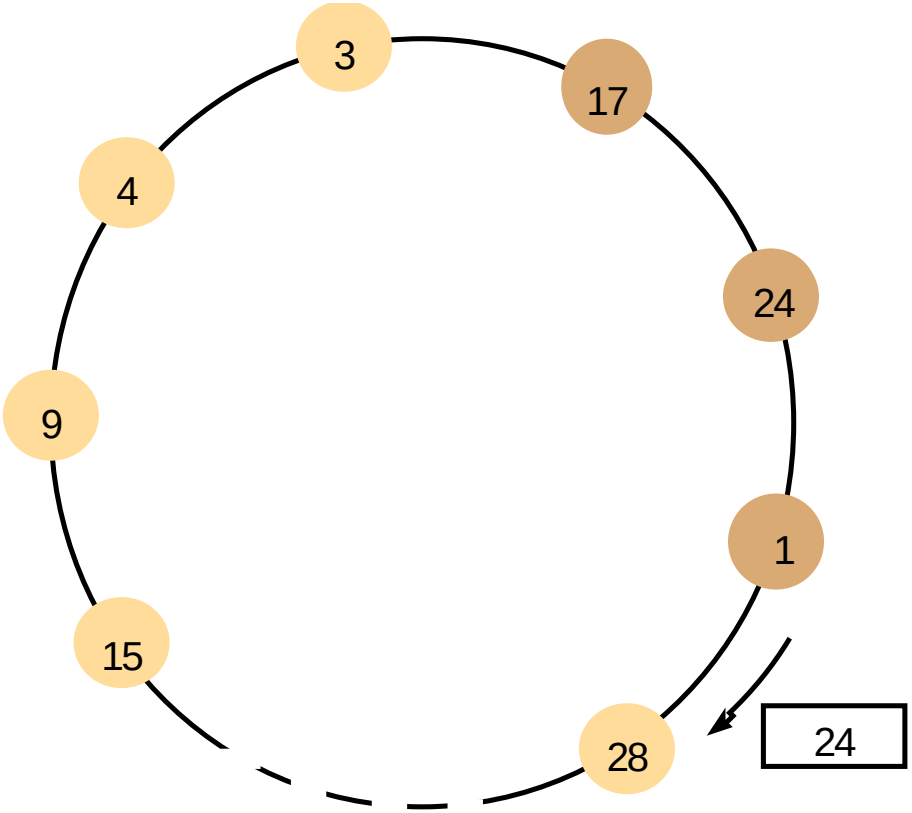
d) Process 6 tells 5 to stop

e) Process 6 wins and tells everyone

Election algorithms

- A ring algorithm (Chang and Roberts):
  - the initiator of the algorithm sends an *election* message, which includes its process identifier, to its successor in the ring
  - when a process receives an election message, it compares its own identifier with that of the message, sending to its successor an election message with the highest identifier
  - when a process receives an election message with its own identifier, it becomes the new coordinator, sending a *leader* message to its successor in the ring
  - the leader message circulates among all the processes in the ring, which adopt it as the new coordinator, until it reaches the coordinator, which does not forward it

A ring-based election in progress



Note: The election was started by process 17.
The highest process identifier encountered so far is 24.
Participant processes are shown darkened.

Election algorithms

- A ring algorithm (Tanenbaum):
  - the initiator of the algorithm sends an *election* message, which includes its process identifier, to its successor
  - when a process *p* receives an election message, it checks if the message contains its own identifier or not
    - if not, *p* includes its identifier and forwards the message
    - otherwise , i.e., if the message contains its identifier, then *p* can determine who is the new coordinator (the process with the highest identifier), and forwards a *leader* message with the same content
    - when the leader message reaches *p* again, i.e., circulates through the whole ring, all processes have determined who is the new coordinator

Group communication

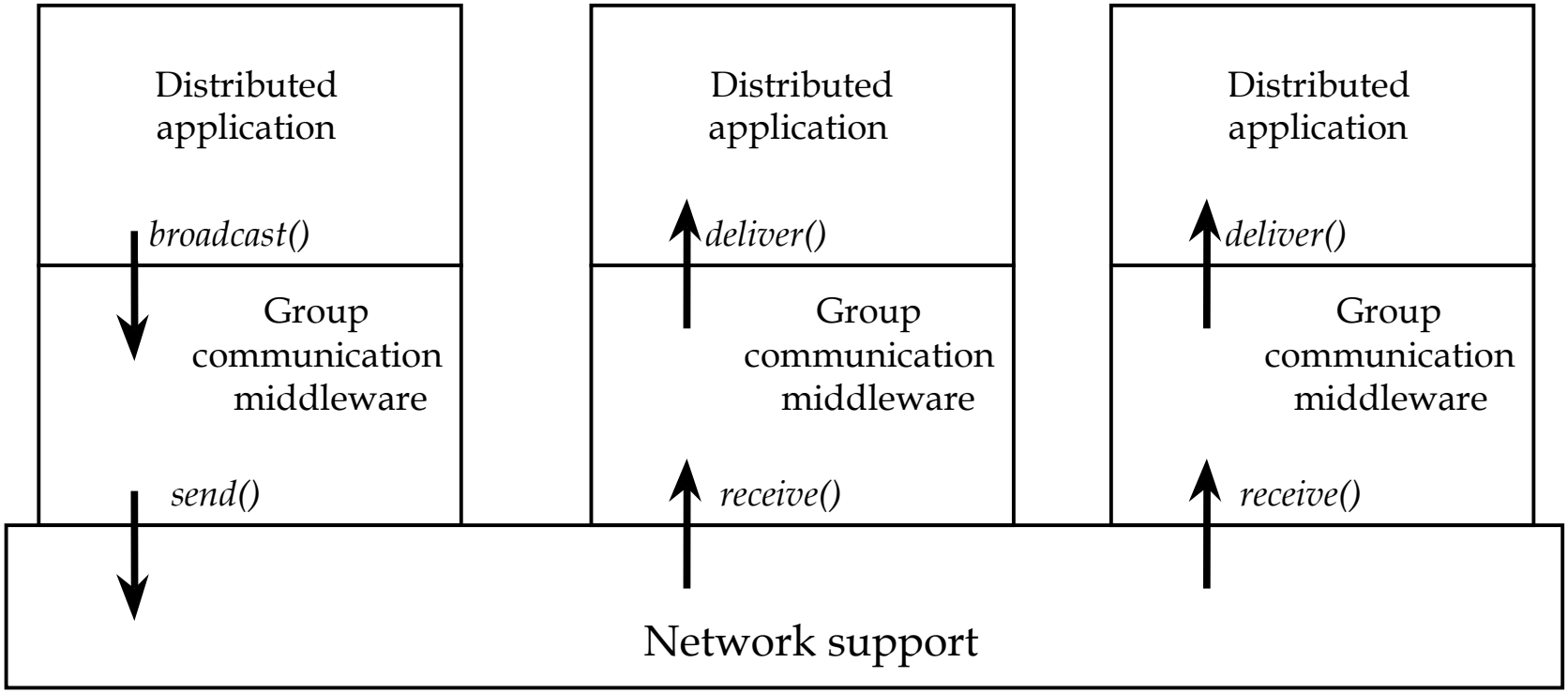
- System model:
  - groups of process
  - diffusion/broadcast/multicast (1:*N* communication), with different semantics
- Examples of applications:
  - event notification
  - registration/search of remote services
  - replication services, for fault-tolerance or high availability/performance
- Examples of systems: *ISIS*, *Horus*, *Ensemble*, *JGroups*, *Transis*, *IP-multicast*, *Spread*, *Appia*

Group communication

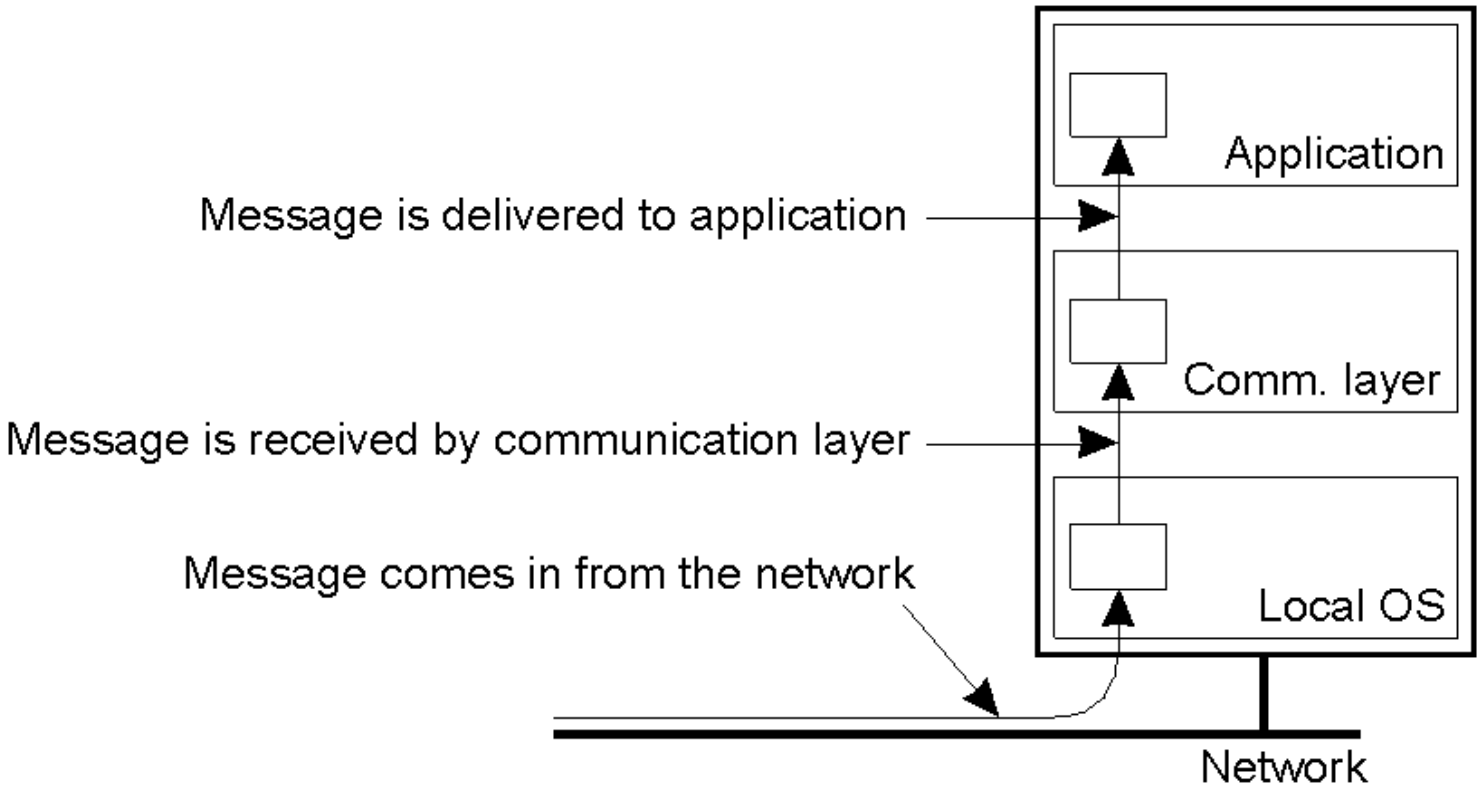
- Group communication support (*middleware)*:
  - identification of groups of processes
  - delivery of messages to the group, with precise reliability and order semantics
  - group membership management (joins, leaves, failures): “synchronous” views
- Communication model:
  - ***broadcast****(G, m)*. Broadcasts message *m* to group *G*. Assumption: the sender is a member of the group
  - ***deliver****(m)*. Delivers message *m* to the process executing the primitive

Group communication

- Communication model:

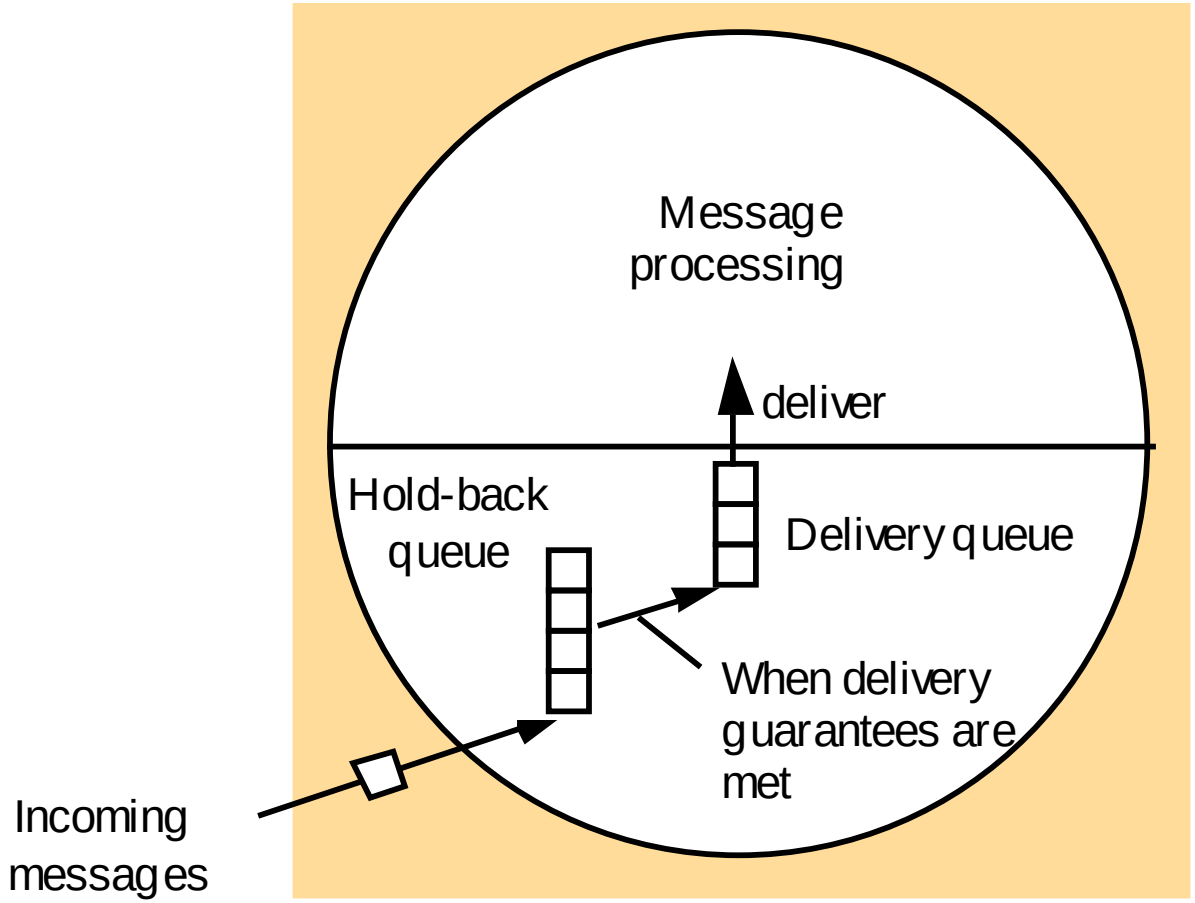


Message Receipt and Delivery



The logical organization of a distributed system to distinguish between message receipt and message delivery

The hold-back queue for arriving multicast messages



Group communication

- Process failure model:
  - processes can fail by permanently crashing
  - two types of processes: correct and incorrect

- Communication channels are quasi-reliable:
  - all the messages sent between correct processes are sooner or later received
  - reliable channels guarantees the reception by correct processes of all the messages sent by incorrect processes (before crashing)
    - used to show theoretical results

Group communication

- Broadcast semantics. Two aspects:
  - reliability
  - order of delivery of the messages

- Basic implementation (unreliable):
  - when executing *broadcast(G, m)*: $\forall p \in G$, *send(p, m)*
  - when executing *receive(m)*: execute *deliver(m)*
    - unreliable broadcast: some correct processes deliver message *m*, while some other correct processes do not deliver it (it can occur if the sender fails while executing *broadcast()*)

Group communication

- Reliable broadcast ("*all or none*"):
  - Validity: if a correct process P_i broadcasts a message *m*, then P_i eventually delivers *m*
  - Agreement: if a correct process delivers a message *m*, then all correct processes eventually deliver *m*
    - unreliable broadcast does not fulfill this property
  - Integrity: for any message *m*, every process delivers *m* at most once, and only if *m* was previously broadcast
- Implementation (*R-broadcast*, *R-deliver*):
  - when P_i executes *R-broadcast(G, m)*: *broadcast(G, m)*
  - when P_j executes *receive(m)* for the first time:
    1. If $P_j \neq P_i$: *broadcast(G, m)*
    2. P_j executes *R-deliver(m)*

Reliable multicast algorithm

On initialization
    Received := {};

For process p to R-multicast message m to group g
    B-multicast(*g*, *m*); // $p \in g$ is included as a destination

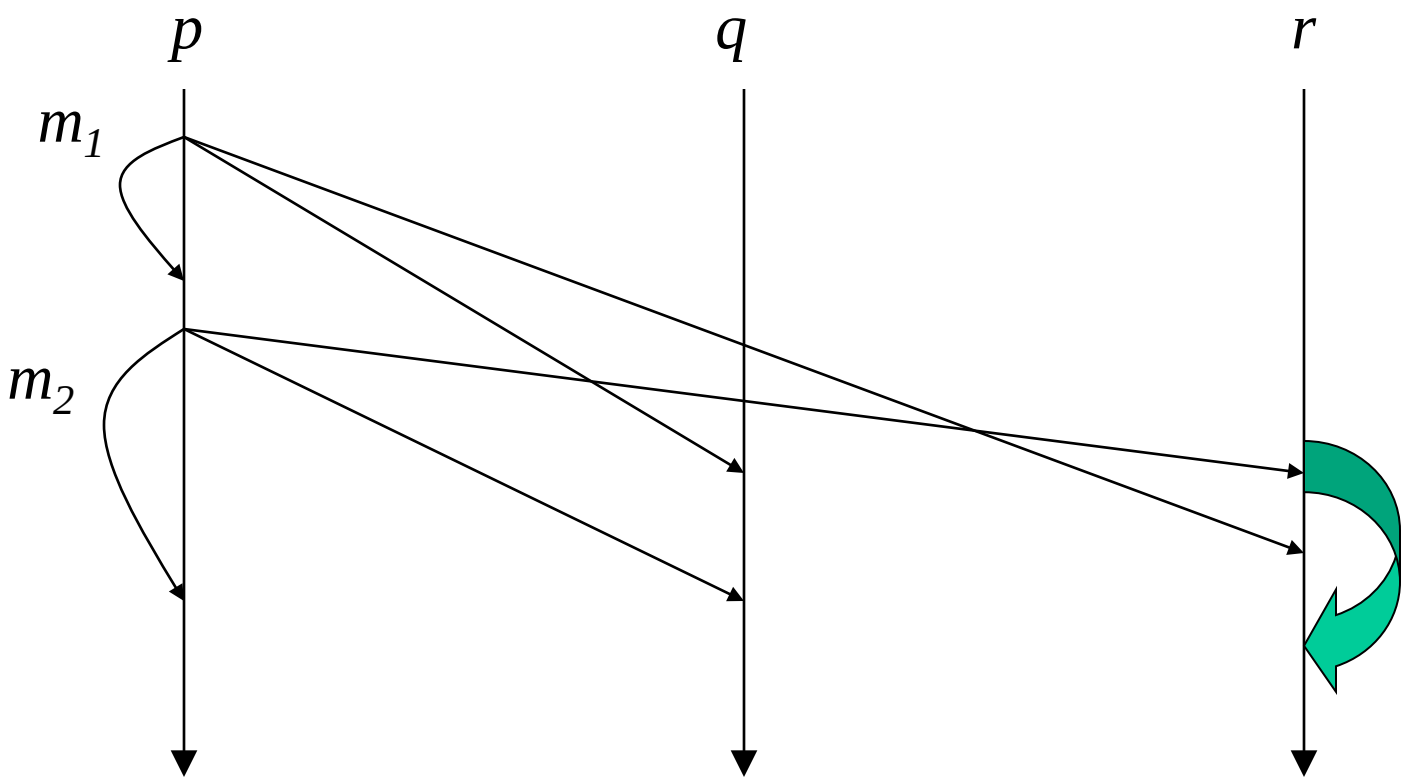
On B-deliver(*m*) *at process q with g = group*(*m*)
    if ($m \notin$ *Received*)
    then
        Received := *Received* $\cup$ {*m*};
        if ($q \neq p$) *then B-multicast*(*g*, *m*); *end if*
        R-deliver m;
    end if

Group communication

- Ordered (reliable) broadcasts:
  - FIFO order: if a process broadcasts a message m_1 before a message m_2, then no correct process delivers m_2 if it has not previously delivered m_1
  - Causal order: if the broadcast of a message m_1 occurs before the broadcast of a message m_2, then no correct process delivers m_2 if it has not previously delivered m_1
    - causal order is stronger than FIFO order
  - Total order: if two correct processes P_i and P_j deliver two messages m_1 and m_2, then P_i delivers m_1 before m_2 if and only if P_j delivers m_1 before m_2
    - also known as atomic broadcast. It is the basis to guarantee consistency in replicated systems

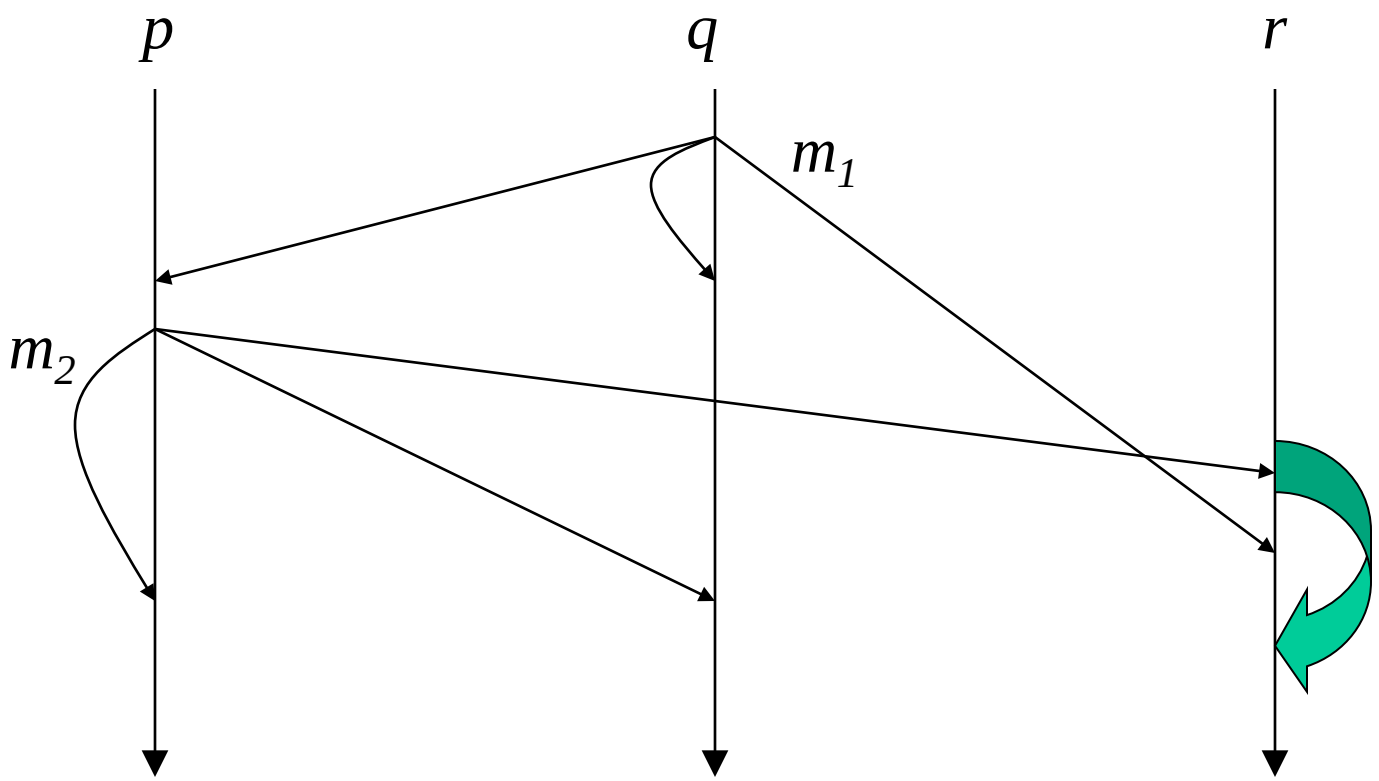
Group communication

- FIFO broadcast:



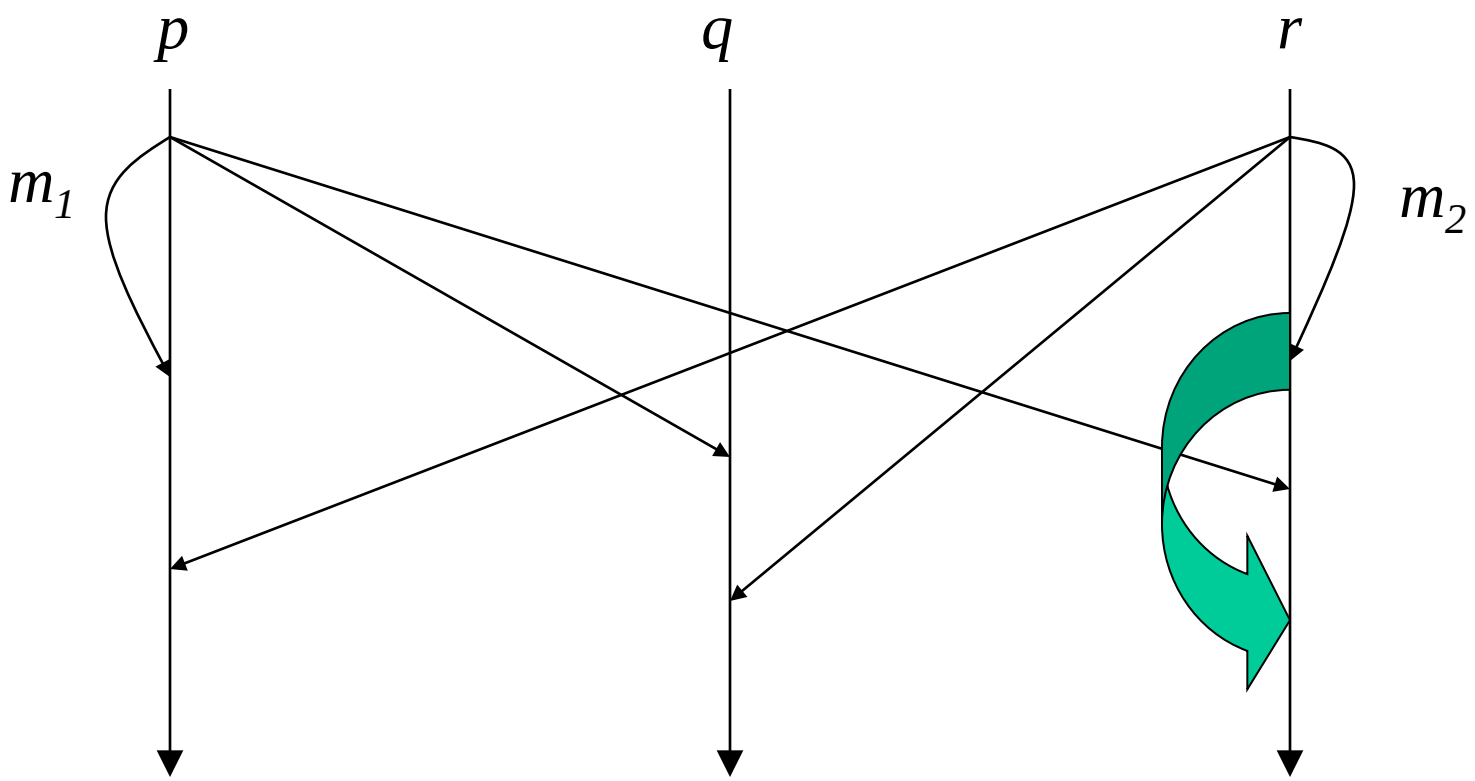
Group communication

- Causal broadcast (causal $\rightarrow$ FIFO):



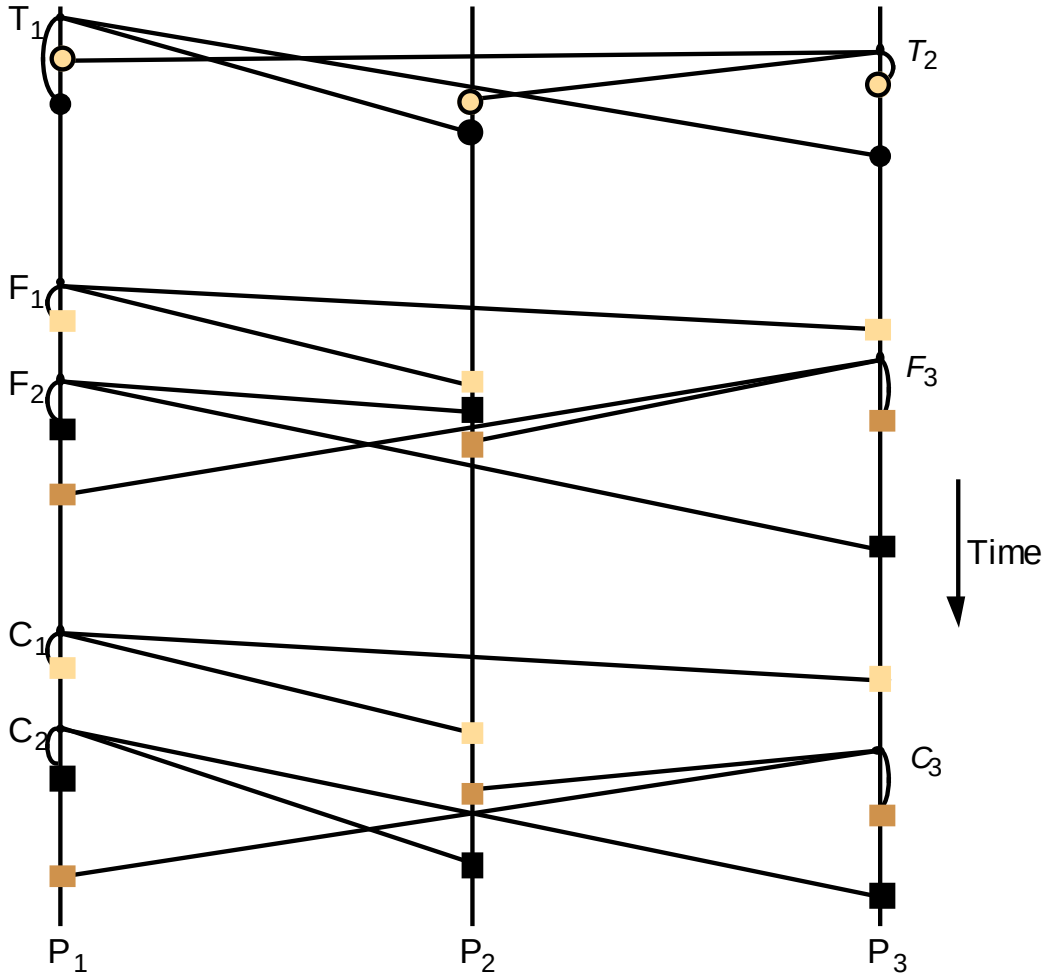
Group communication

- Total order broadcast:



Total, FIFO and causal ordering of multicast messages

Notice the consistent ordering of totally ordered messages T_1 and T_2,
the FIFO-related messages F_1 and F_2 and the causally related messages C_1 and C_3
– and the otherwise arbitrary delivery ordering of messages



Group communication

- Broadcast types:
  - unreliable broadcast (for example, *IP-multicast*)
  - reliable broadcast
    - unordered
    - FIFO broadcast
    - causal broadcast (thus FIFO)
    - total order broadcast
    - FIFO and total order broadcast
    - causal and total order broadcast (thus FIFO)
- Example: *USENET News*
    - reliable (*TCP*), FIFO (non causal, non total)
    - causal in practice: insert the original message in the reply

Display from bulletin board system

Alice

Bulletin board: *os.interesting*		
Item	From	Subject
23	A.Hanlon	Mach
24	G.Joseph	Microkernels
25	A.Hanlon	Re: Microkernels
26	T.L'Heureux	RPC performance
27	M.Walker	Re: Mach
end		

Bob

Bulletin board: *os.interesting*		
Item	From	Subject
23	A.Hanlon	Mach
24	T.L'Heureux	RPC performance
25	M.Walker	Re: Mach
26	A.Hanlon	Re: Microkernels
27	G.Joseph	Microkernels
end		

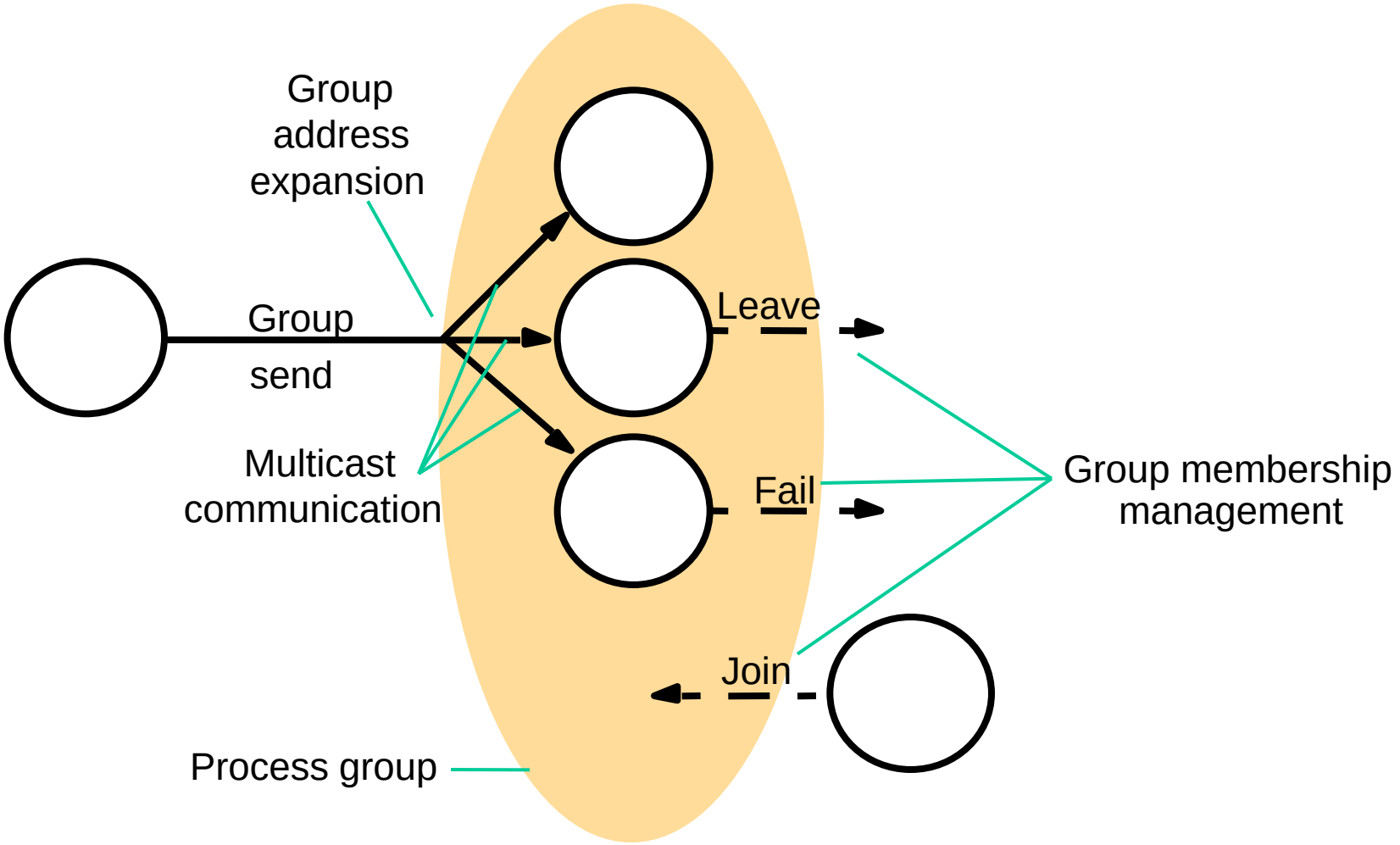
Group communication

- Implementation of ordered broadcasts:
  - FIFO broadcast: number messages in the sender
  - causal broadcast: using time vectors
  - total order: timed reliable broadcast (synchronous) :
    - when P_i executes *TO-broadcast(G, m)*: *R-broadcast(G, m)*, labeling *m* with the local timestamp *t(m)*
    - when P_j executes *R-deliver(m)*: schedule *TO-deliver(m)* for the local time instant $t(m)+\Delta$
      - Δ: constant based on the maximum delay of messages, the maximum number of processes that can fail and the maximum deviation among clocks
      - if *TO-deliver* cannot be executed at the scheduled time instant (because the message is *R-delivered* too late), the system does not behave synchronously

Group communication

- Membership management (dynamic groups):
  - management of leaves (due to failure) and joins (due to recovery). Operations:
    - ***join***(P_i, G)
    - ***leave***(P_i, G)
  - a failure detection mechanism is needed
  - *view*: set of current processes of the group
    - when executing *join()* or *leave()*, a view change occurs
    - a group is modeled as a sequence of views: $v_0, v_1, ..., v_k$
    - operation for installing a new view: ***install***(v_{k+1})
    - a protocol for consistent view management is needed
    - given a view, a coordinating process can be directly elected

Services provided for process groups

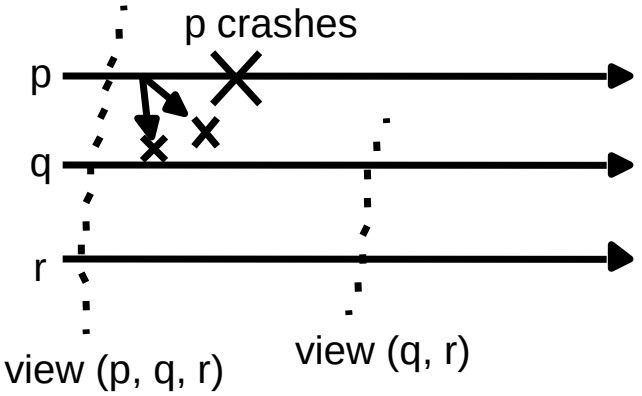


Group communication

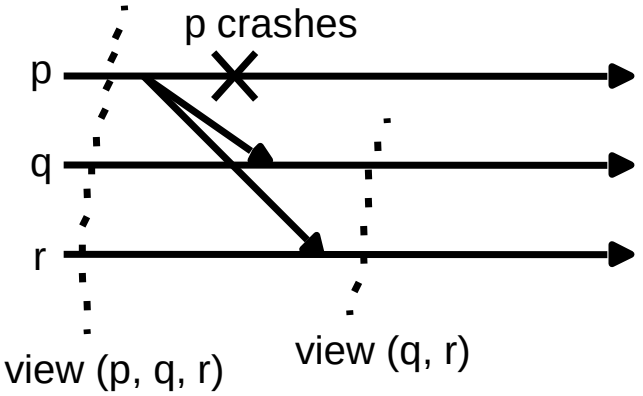
- View management properties:
  - Termination: the failure of a process or the execution of *join()* or *leave()* leads sooner or later to the installation of a new view.
  - Agreement: all the correct processes deliver the same set of messages in a given view.
  - Validity: Let P_i be a correct process that delivers a message m in a view v_k. If some process $P_j \in v_k$ does not deliver m in v_k, then P_j will not be in the view v_{k+1} installed by P_i.

View-synchronous group comm.

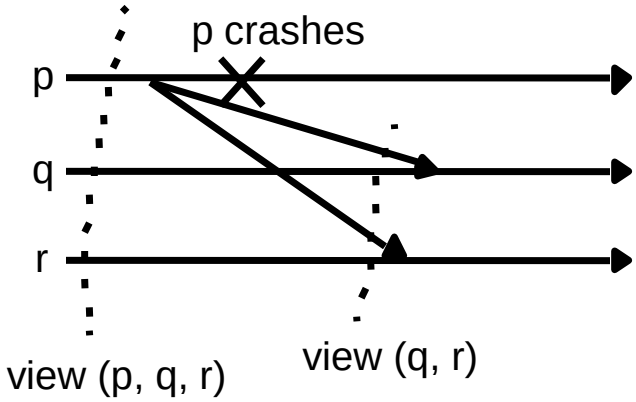
a (allowed).



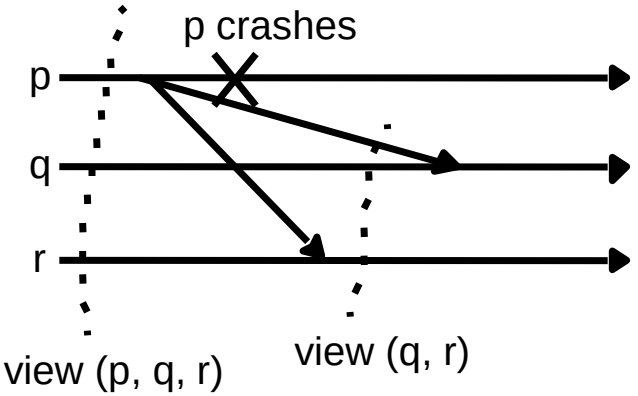
b (allowed).



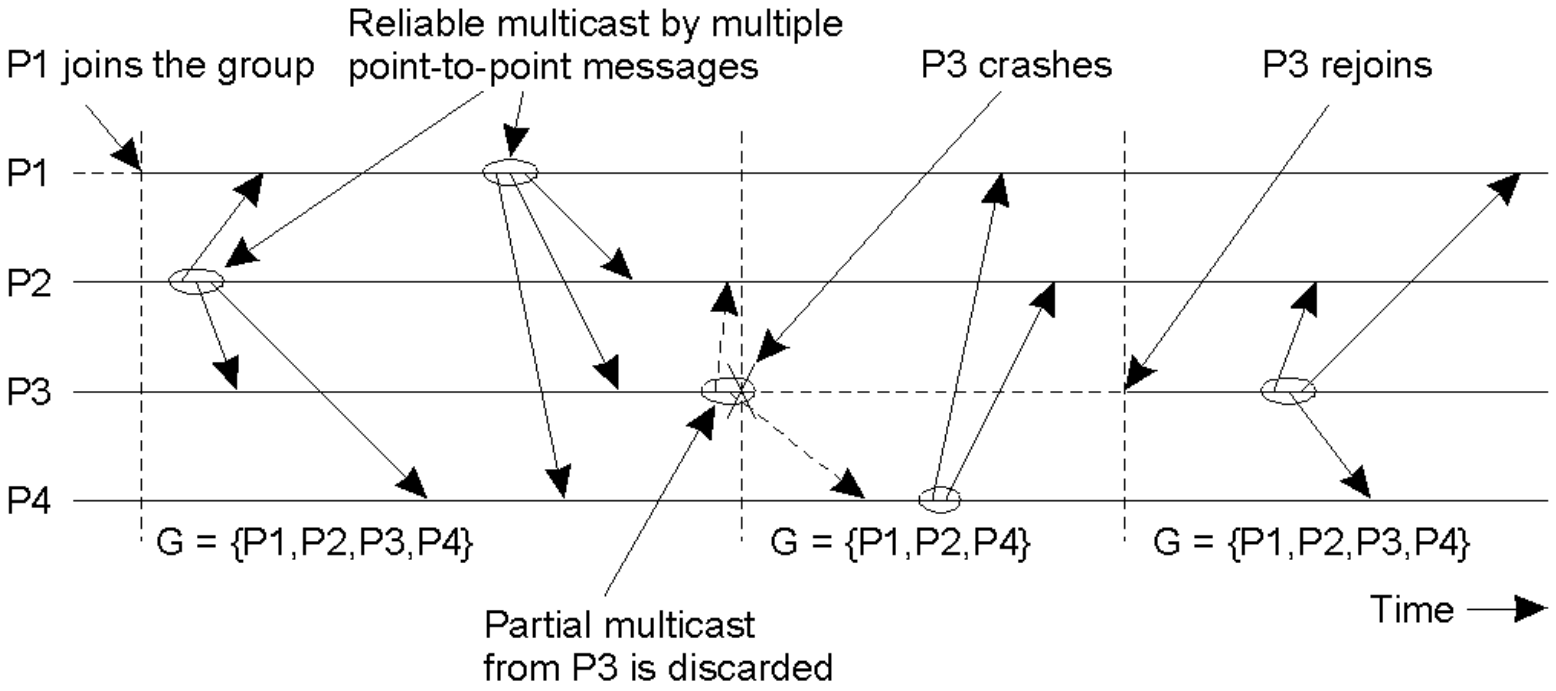
c (disallowed).



d (disallowed).

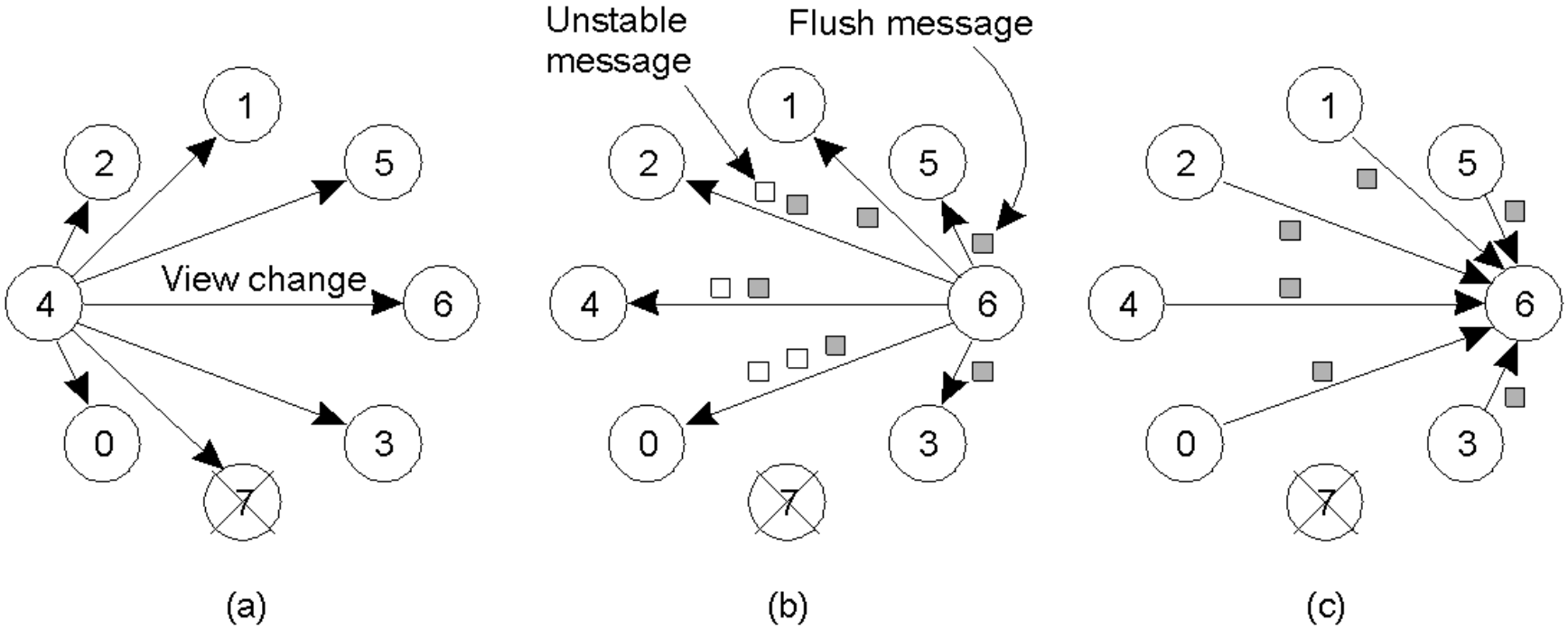


View Synchrony



The principle of view synchronous multicast

Implementing View Synchrony



a) Process 4 notices that process 7 has crashed, sends a view change

b) Process 6 sends out all its unstable messages, followed by a flush message

c) Process 6 installs the new view when it has received a flush message from everyone else

Group communication

- ISIS system:
  - group communication middleware
  - developed for UNIX (Cornell University, NY, USA)
  - process groups + view management + broadcasts
  - *FBCAST*: FIFO broadcast
  - *CBCAST*: causal broadcast
    - based on time vectors
  - *ABCAST*: causal and (atomic) total order broadcast
    - based on a coordinating (sequencer) process
  - *GBCAST*: view management
    - *flush* protocol

Causal ordering using vector timestamps (ISIS)

Algorithm for group member p_i ($i = 1, 2\ldots, N$)

On initialization

$V_i^g[j] := 0$ ($j = 1, 2\ldots, N$);

To CO-multicast message m to group g

$V_i^g[i] := V_i^g[i] + 1$;

B-multicast(g, $<V_i^g, m>$);

On B-deliver($<V_j^g, m>$) *from* p_j, *with* $g = group(m)$

place $<V_j^g, m>$ in hold-back queue;

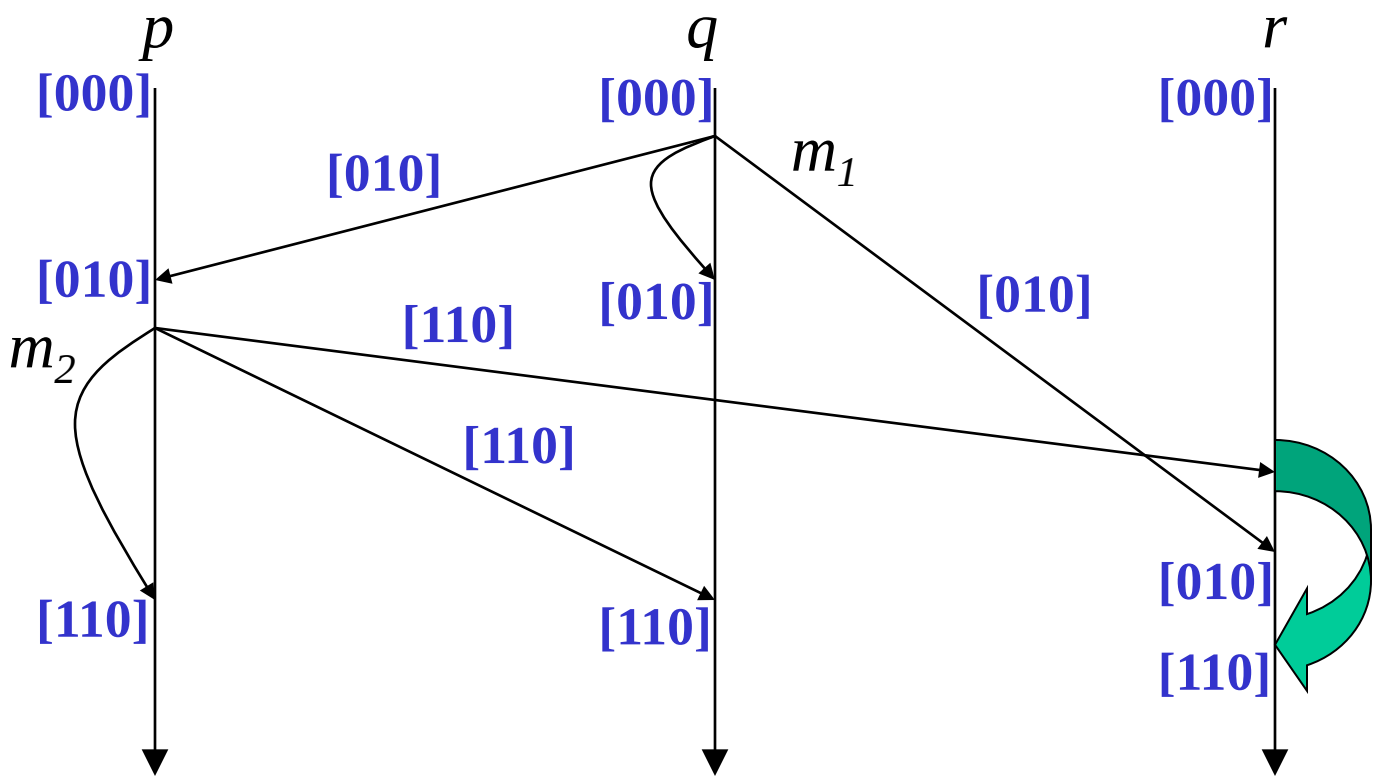
wait until $V_j^g[j] = V_i^g[j] + 1$ and $V_j^g[k] \leq V_i^g[k]$ ($k \neq j$);

CO-deliver m; // after removing it from the hold-back queue

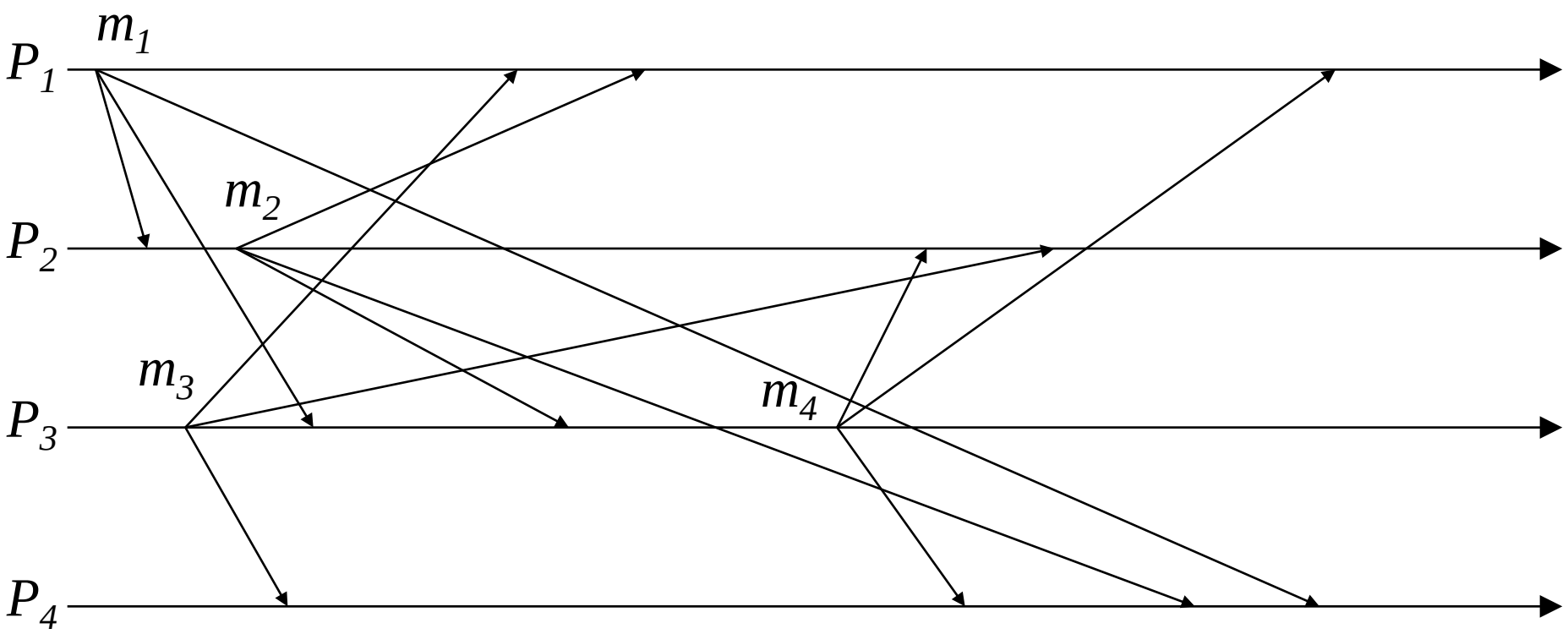
$V_i^g[j] := V_i^g[j] + 1$;

Group communication

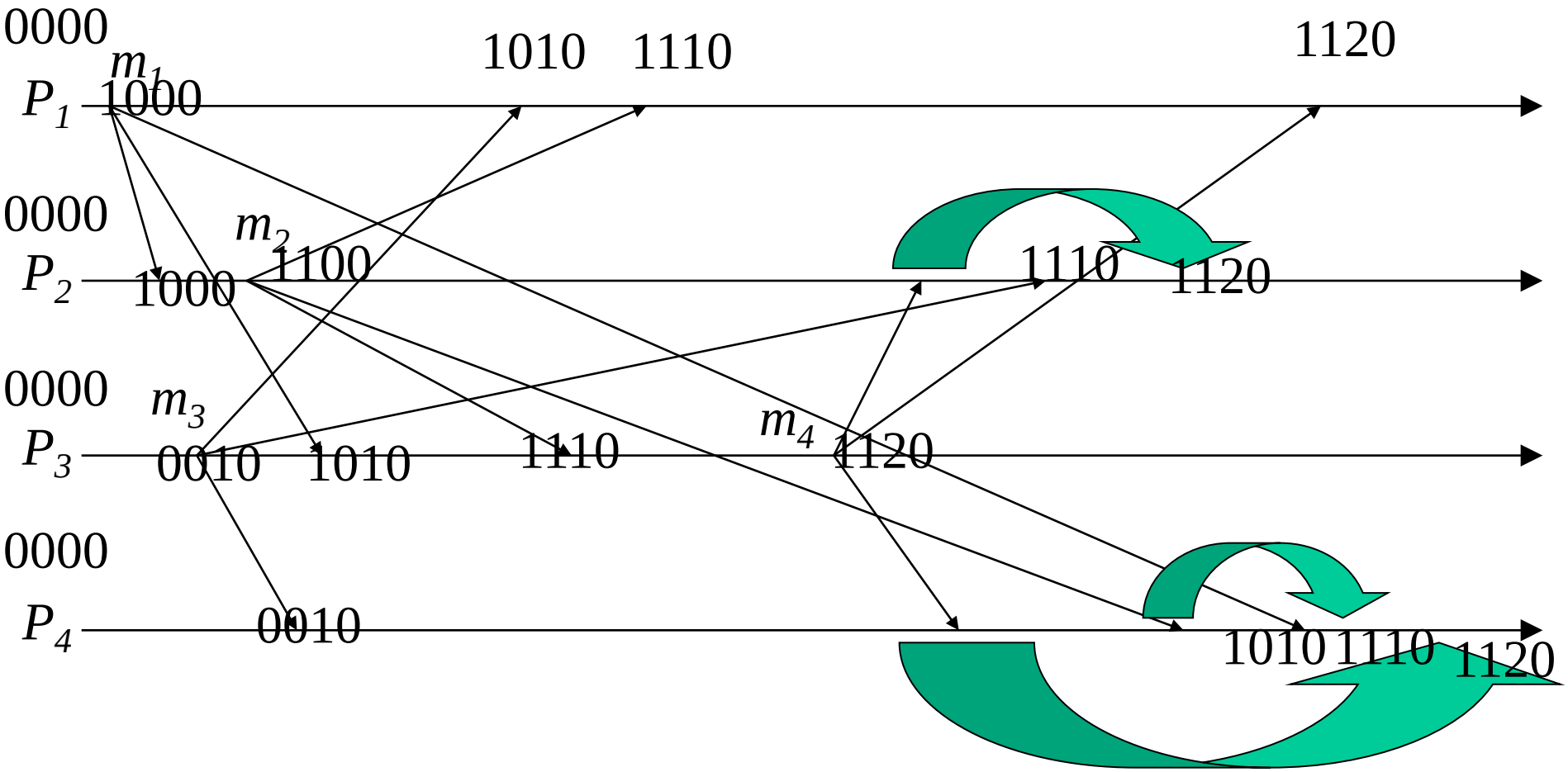
- Example of causal broadcast:



Causal broadcast. Exercise



Causal broadcast. Exercise



Total ordering using a sequencer

1. Algorithm for group member p

On initialization: $r_g := 0$;

To TO-multicast message m to group g
    B-multicast($g \cup \{sequencer(g)\}$, <*m, i*>);

On B-deliver(<*m, i*>) *with* $g = group(m)$
    Place <*m, i*> in hold-back queue;

On B-deliver(m_{order} = <"order", *i, S*>) *with* $g = group(m_{order})$
    wait until <*m, i*> in hold-back queue and $S = r_g$;
    TO-deliver m; // (after deleting it from the hold-back queue)
    $r_g = S + 1$;

2. Algorithm for sequencer of g

On initialization: $s_g := 0$;

On B-deliver(<*m, i*>) *with* $g = group(m)$
    B-multicast(g, <"order", *i*, s_g>);
    $s_g := s_g + 1$;

Replication

(1) Consistent replication:

- main goal: fault tolerance
- each node manages a complete copy (distributed management)
- nodes are close from each other (e.g., in a local area network)
- guaranteeing consistency is essential
- example: bank account management systems

(2) Distribution of a service:

- main goal: availability/scalability, reduce latency
- partial or total replication (centralized management)
- nodes are geographically distributed
- guaranteeing consistency is secondary
- example: *mirroring* on the Internet

Replication

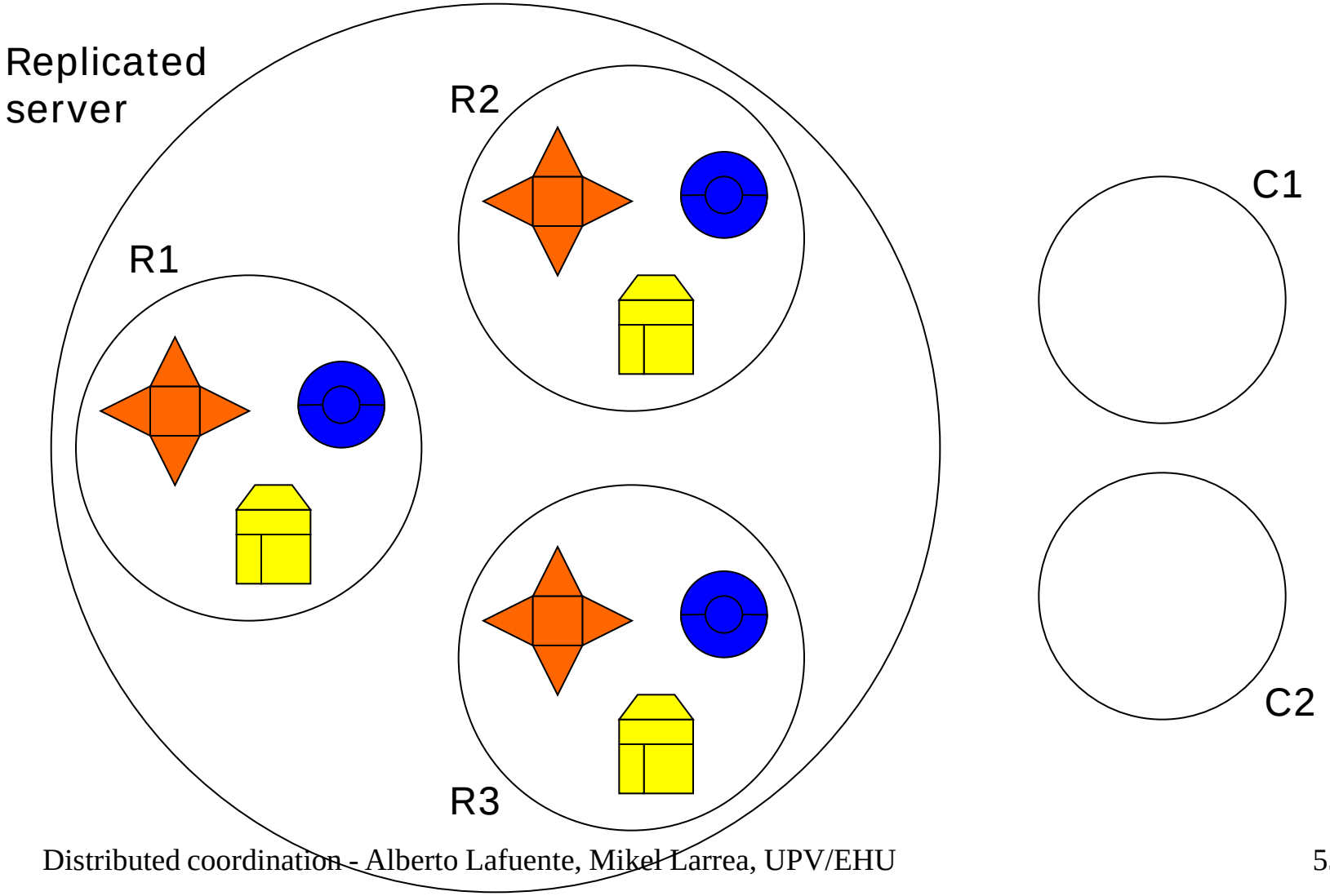
(3) *Caching*:

- storing recent information locally (or close)
- main goal: improve performance, reduce latency
- partial replication (management in the client)
- temporary storage
- guaranteeing consistency is important, but not essential
- examples: *Web*, distributed file systems (e.g., *NFS*)

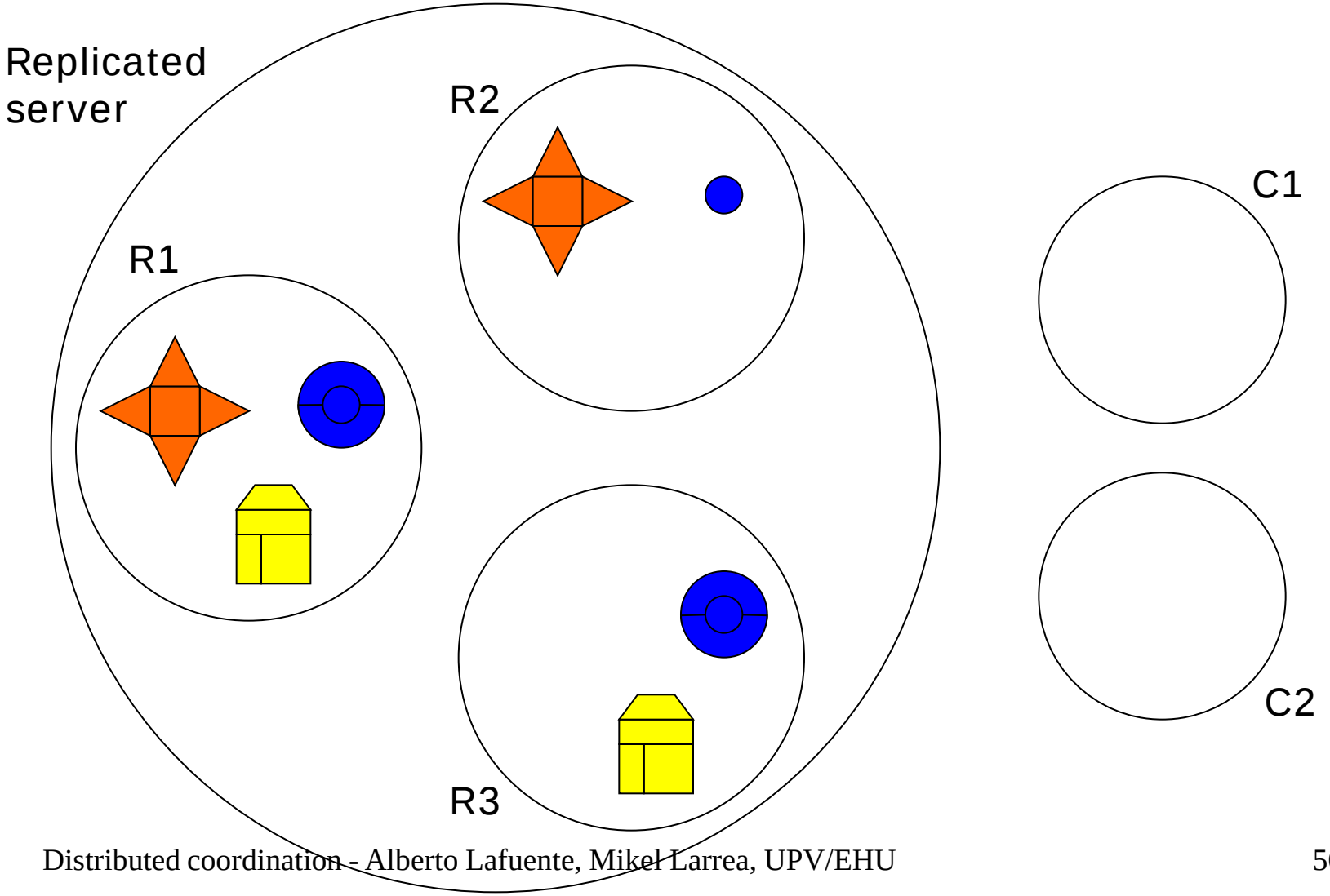
– Replication types can be combined (e.g., *Web*):

- replicated servers in a local area network
- geographically remote copies (*mirroring*)
- clients store locally copies of recently accesses pages (*caching*)
- *proxy* nodes (hierarchical *caching*)

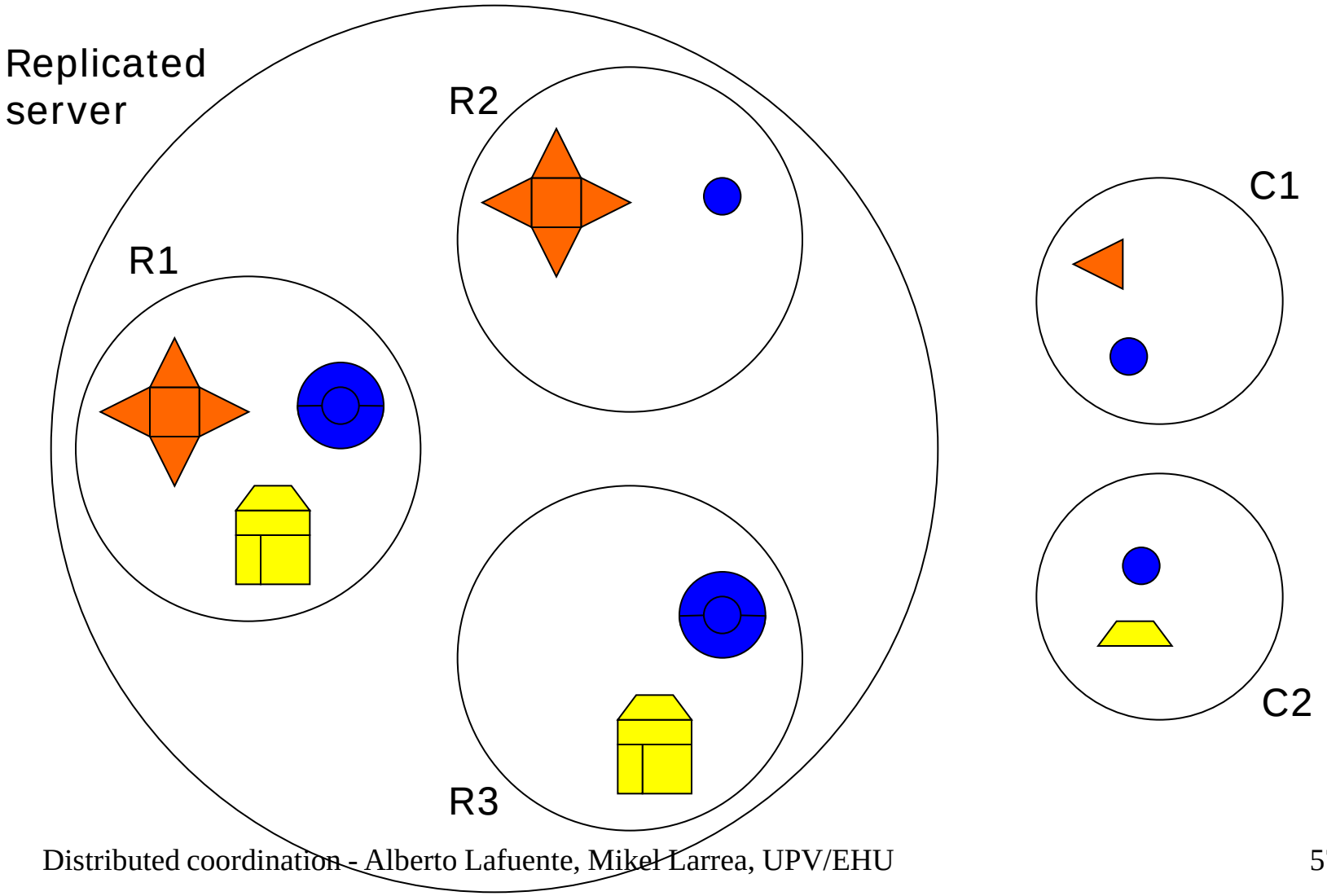
Consistent replication



Distribution of a service



Caching

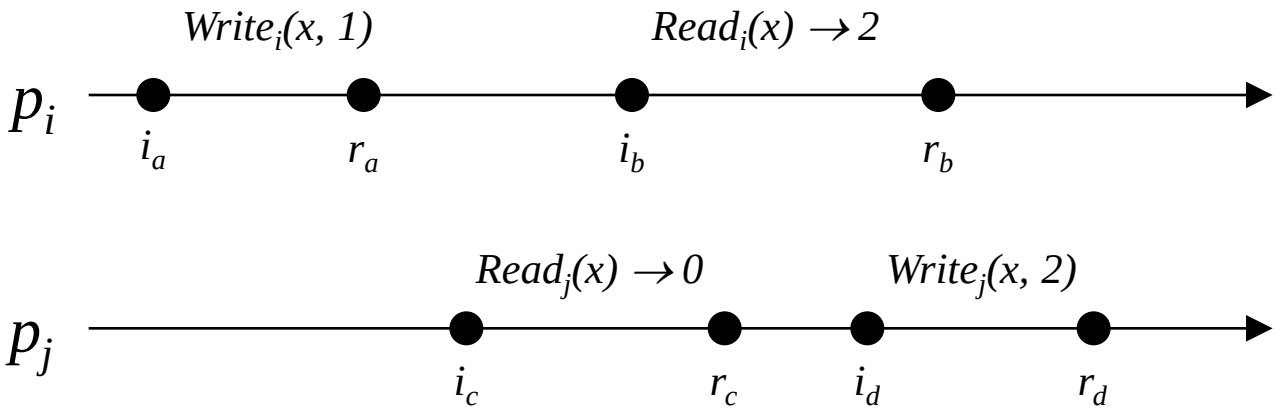


Consistent replication

- System model:
  - clients make requests to the server
    - in principle, the server can be replicated or not
  - each operation o_k consists of two events: (i_k , r_k)
    - *invocation*: i_k , *reply*: r_k
  - clients are synchronous (sequential): given a sequence of operations of the same client [o_1 , o_2 , ..., o_k , ...], i_{k+1} only occurs after r_k
  - example: variable *x*
    - operations: *Read(x)* $\rightarrow$ *v*, *Write(x, v)*
    - usual semantics: *Read(x)* returns the last value written in the variable, *Write(x, v)* writes value *v* in the variable

Consistent replication

- Example (initially, $x = 0$):



  - operation sequence: p_i: [o_a , o_b], p_j: [o_c , o_d]
  - global event sequence: $\tau = [i_a , r_a , i_c , i_b , r_c , i_d , r_b , r_d]$
  - by permuting the events of τ we obtain executions

    legal: σ_1 = [*Read$_j$(x) → 0, Write$_i$(x, 1), Write$_j$(x, 2), Read$_i$(x) → 2*]

    illegal: σ_2 = [*Write$_i$(x, 1), Read$_j$(x) → 0, ...*]

Consistent replication

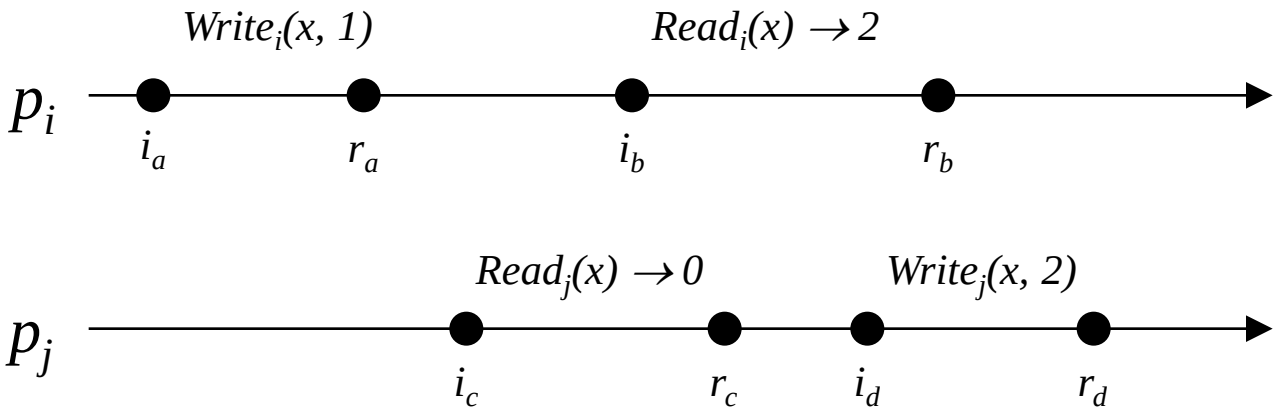
- Types of consistency (based on the conditions to obtain legal executions from τ):
  - sequential consistency: a legal execution σ has sequential consistency with respect to τ if the order of operations for each process is the same in σ and in τ
    - e.g.: [$Read_j(x) \rightarrow 0$, $Write_i(x, 1)$, $Write_j(x, 2)$, $Read_i(x) \rightarrow 2$]
    - the real time order is not respected: although r_a occurs before i_c, in τ, in σ o_c is executed before o_a
      - this situation can occur in a replicated service, for example if a non updated copy is read
    - in some applications sequential consistency is not enough
      - example: in a bank account management, the order of deposits and withdrawals cannot be changed (the final state may differ)

Consistent replication

- Types of consistency:
  - linearizability: a legal execution σ is linearizable if, in addition to being sequentially consistent, given any pair of operations o_k o_h, if r_k occurs before i_h, in τ, then in σ o_k is executed before o_h
    - to be linearizable, the real time order must be respected. In the example, $Write_i(x, 1)$ must be executed before $Read_j(x)$, because r_a occurs before i_c in τ
    - [$Read_j(x) \rightarrow 0$, $Write_i(x, 1)$, $Write_j(x, 2)$, $Read_i(x) \rightarrow 2$]: NO
    - [$Write_i(x, 1)$, $Read_j(x) \rightarrow 1$, $Write_j(x, 2)$, $Read_i(x) \rightarrow 2$]: YES
      - to be legal, $Read_j(x) \rightarrow 0$ has been replaced by $Read_j(x) \rightarrow 1$
    - [$Write_i(x, 1)$, $Read_j(x) \rightarrow 1$, $Read_i(x) \rightarrow 1$, $Write_j(x, 2)$]: YES
      - to be legal, $Read_i(x) \rightarrow 2$ has been replaced by $Read_i(x) \rightarrow 1$ too

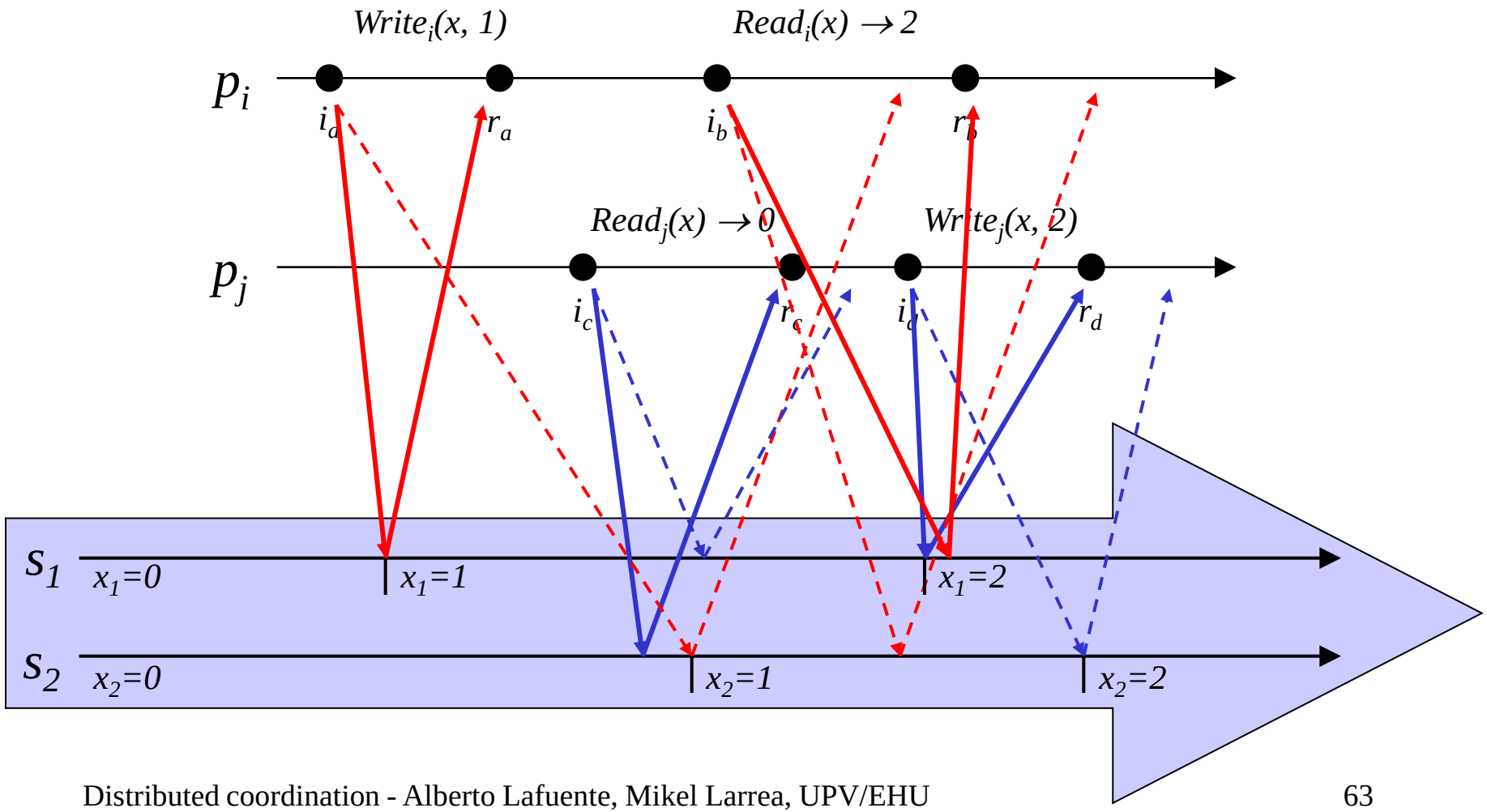
Consistent replication

- Summary (initially, $x = 0$):



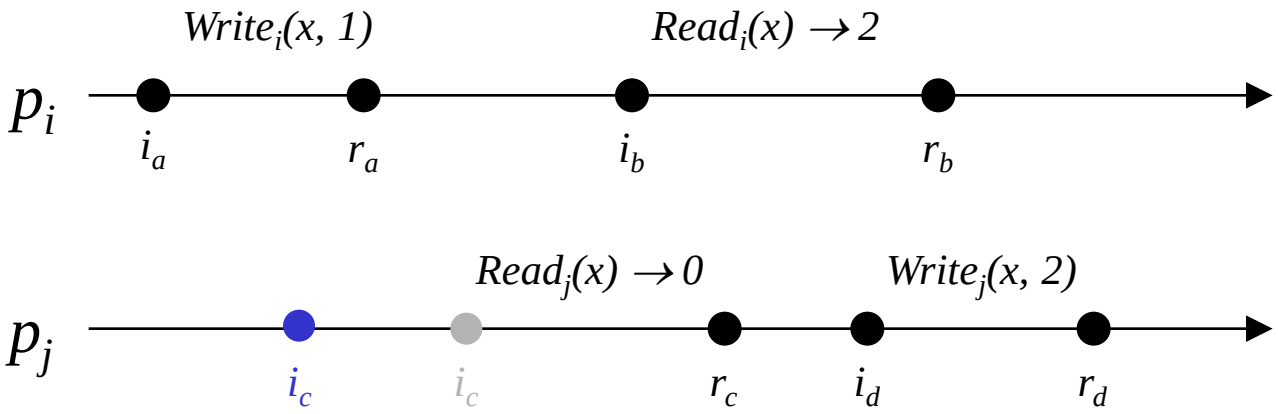
  - global sequence of events: $\tau = [i_a, r_a, i_c, i_b, r_c, i_d, r_b, r_d]$
  - illegal execution: $\sigma_1 = [Write_i(x, 1), Read_j(x) \rightarrow 0, ...]$
  - legal execution: $\sigma_2 = [Read_j(x) \rightarrow 0, Write_i(x, 1), Write_j(x, 2), Read_i(x) \rightarrow 2]$
    - is sequentially consistent
    - is not linearizable
  - linearizable executions:
    - $\sigma_3 = [Write_i(x, 1), Read_j(x) \rightarrow 1, Write_j(x, 2), Read_i(x) \rightarrow 2]$
    - $\sigma_4 = [Write_i(x, 1), Read_j(x) \rightarrow 1, Read_i(x) \rightarrow 1, Write_j(x, 2)]$

Consistent replication



Consistent replication

- Another example (initially, $x = 0$):



  - global sequence of events: $\tau' = [i_a, i_c, r_a, i_b, r_c, i_d, r_b, r_d]$
  - execution σ_2: $[Read_j(x) \rightarrow 0, Write_i(x, 1), Write_j(x, 2), Read_i(x) \rightarrow 2]$
    - legal
    - sequentially consistent
    - **linearizable!**

Consistent replication

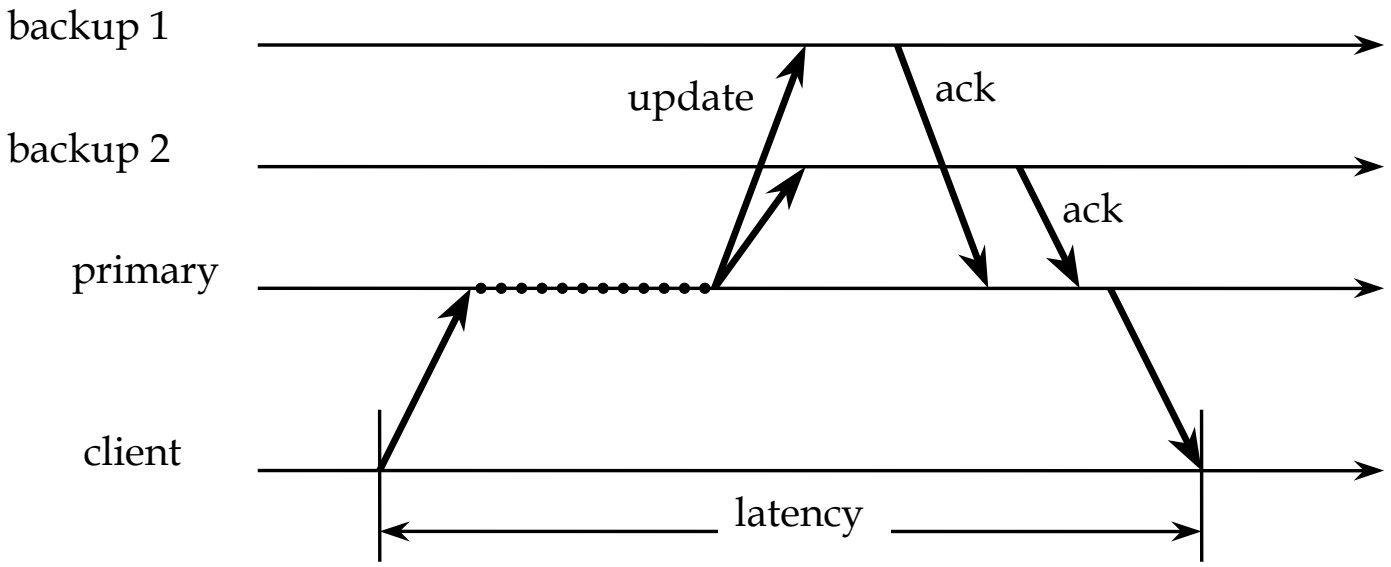
- Linearizability guarantees transparency in replicated systems. The following two conditions are sufficient to ensure linearizability:

  (1) order: all replicas execute requests in the same order

  (2) atomicity ("all or none"): if a replica executes a request, then all the correct replicas execute the request sooner or later

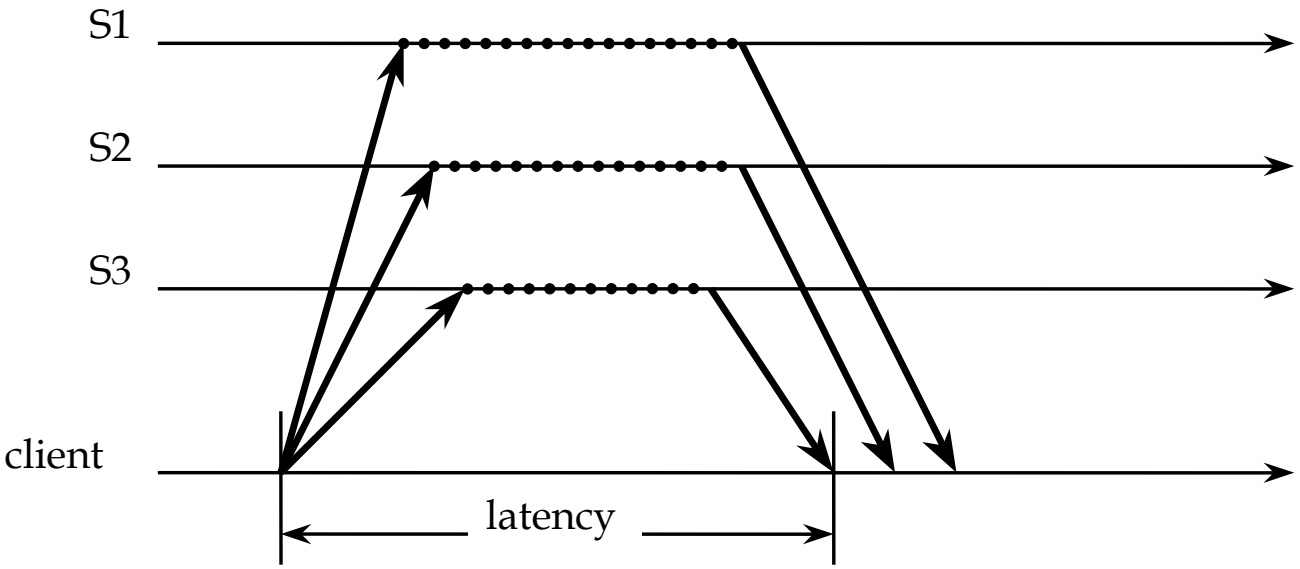
- Replication techniques:
  - passive replication (*primary-backup*)
  - active replication

Passive replication



- order: defined by the primary
- atomicity: the update of the backups must be atomic
- if the primary fails:
  - election of a new primary (view change)
  - the client may resend its request. It must be guaranteed that the request is only processed once!

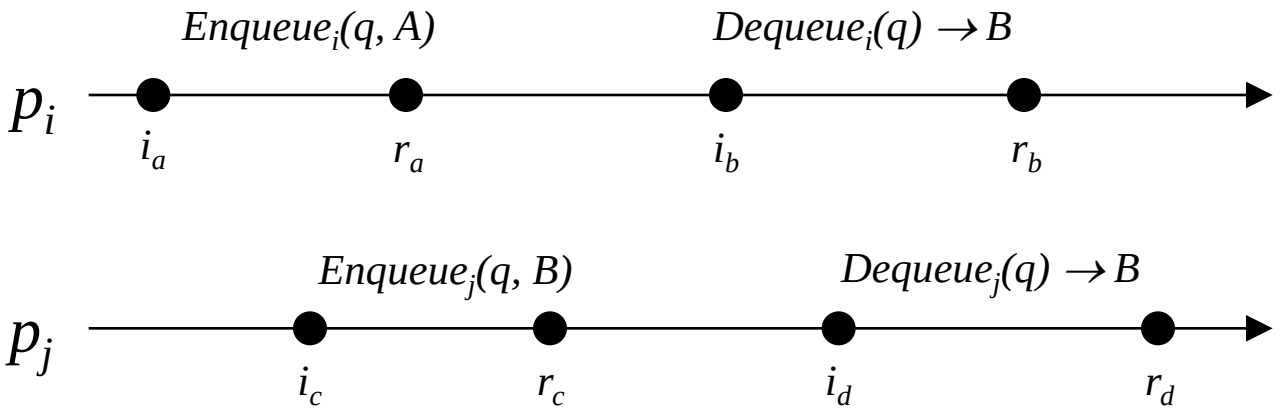
Active replication



- order and atomicity: client requests are made through total order broadcast (*TO-broadcast*)
- the client waits for the first reply
- the failure of a replica is completely transparent to the client
- in theory it has a lower latency than passive replication
  - but total order broadcast is more “expensive”

Consistent replication

- Exercise:

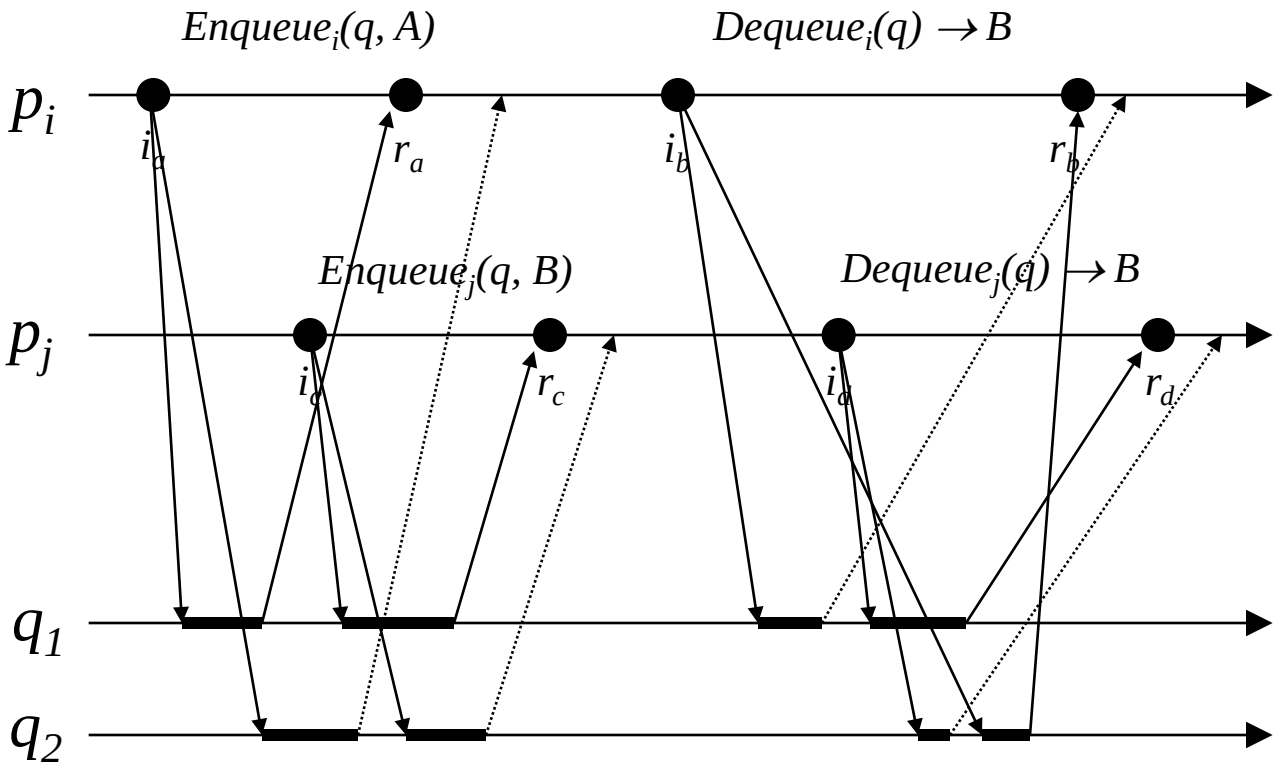


  - q is a replicated queue (2 replicas)
  - type of communication between the client and each replica:
    - reliable and FIFO

  - What type of consistency has the execution?
  - How can this be explained?

Consistent replication

- Exercise (interpretation):



Distributed transactions

- Transaction: sequence of operations that consistently modifies the state of the system
  - used for long in the context of databases to control concurrent access to data
    - if the database is distributed, transactions will be distributed. In addition, if the database is replicated, consistency among replicas must be guaranteed
    - performance (*throughput*) is a primary goal
    - example (*a* and *b* are bank accounts):

```
BEGIN_TRANSACTION
   a.withdraw(1000€)
   b.deposit(1000€)
END_TRANSACTION
```

Distributed transactions

- Transactions must fulfill the *ACID* properties:
  - *Atomicity* (“all or nothing”): a transaction completes satisfactorily (*commit*) or has no effect on the state of the system (*abort*)
  - *Consistency*: the execution of a transaction preserves the invariants of the system state
  - *Isolation*: the execution of a transaction is independent of the concurrent execution of other transactions
  - *Durability*: the effects of a transaction survive future failures

Distributed transactions

- System model:
  - when a client wants to execute a transaction, it indicates the sequence of operations to be performed
  - operations:
    - BEGIN_TRANSACTION: beginning of a transaction
    - END_TRANSACTION: end of a transaction, initiating the attempt to commit it
    - ABORT_TRANSACTION: abort the transaction, restoring the global state previous to its execution
    - READ_OBJECT / WRITE_OBJECT: object manipulation
  - the transaction is broadcast to all the involved nodes
    - a synchronous model is assumed

The Transaction Model

```
BEGIN_TRANSACTION
  reserve WP -> JFK;
  reserve JFK -> Nairobi;
  reserve Nairobi -> Malindi;
END_TRANSACTION
```

(a)

```
BEGIN_TRANSACTION
  reserve WP -> JFK;
  reserve JFK -> Nairobi;
  reserve Nairobi -> Malindi full =>
ABORT_TRANSACTION
```

(b)

a) Transaction to reserve three flights commits
b) Transaction aborts when third flight is unavailable

Distributed transactions

- System model (cont.):
  - each node/participant, upon reception of the transaction, tries to carry out its corresponding operations. If it succeeds, its vote will be favorable to commit (YES). Otherwise, its vote will be unfavorable (NO)
  - finally, all participants execute an atomic commitment protocol, in order to agree on a decision for the transaction (*commit* / *abort*)
    - if the decision is *commit*, then all the participants update their state
    - if the decision is *abort*, then no participant updates its state

Distributed transactions

- Properties of atomic commitment protocols:
  - (1) all participants that decide will do so in the same way
  - (2) each participant decides at most once (decisions are irreversible)
  - (3) if a participant decides *commit*, then all participants have voted YES
  - (4) if all the participants vote YES and there is no failure, then all the participants decide *commit*
- Example: Two Phase Commit (2PC)

Distributed transactions

- Two Phase Commit:
  - First phase:
    - the coordinator sends a vote request to all the participants
    - each participant waits for the message from the coordinator. If the message does not arrive (the coordinator has failed) it decides *abort*. Otherwise, it sends its vote to the coordinator
    - if all the votes are YES, the coordinator decides *commit*. Otherwise, if some vote is NO or some participant does not reply, the coordinator decides *abort*
  - Second phase (assuming that the coordinator is correct):
    - the coordinator sends its decision to all the participants
    - each participant, upon reception of the decision, decides too

Distributed transactions

- Two Phase Commit (without failures):

```
if pi = coordinator then send <Vote?> to all
when (received <Vote?> from coordinator)
   send <vi,Vote> to coordinator
if pi = coordinator then
   wait until (received <vj,Vote> from every pj)
   if (received <no,Vote> from some pj) then
      decision := abort
   else
      decision := commit
   send <decision,Decision> to all
wait until (received <v,Decision> from coordinator
if pi ≠ coordinator then
   decision := v
```

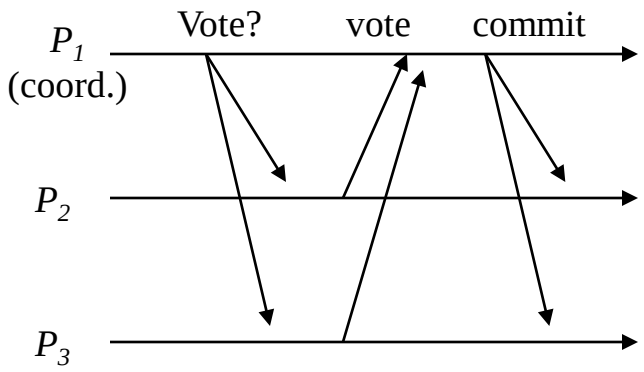
Distributed transactions

- Distributed Two Phase Commit (D2PC, without failures):

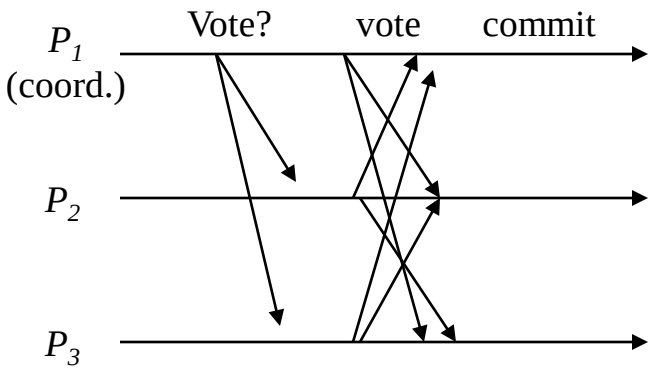
```
if pi = coordinator then send <Vote?> to all
when (received <Vote?> from coordinator)
   send <vi,Vote> to all
wait until (received <vj,Vote> from every pj)
if (received <no,Vote> from some pj) then
   decision := abort
else
   decision := commit
```

Distributed transactions

- (Distributed) Two Phase Commit (2PC, D2PC):



2PC

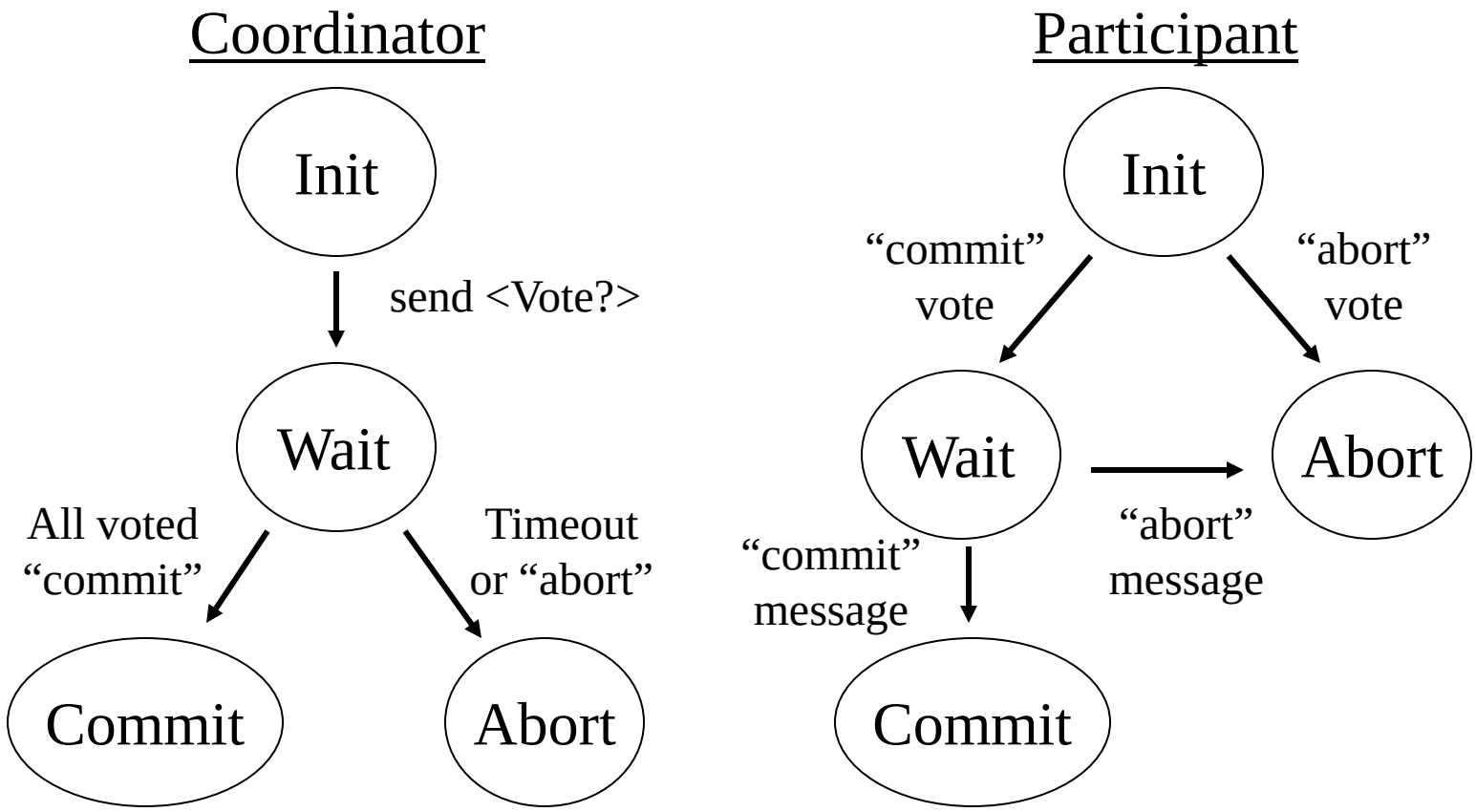


D2PC

- Exercises:
  - (1) verify that both protocols are correct, i.e., they satisfy the atomic commitment properties
  - (2) compare the protocols: number of messages (without and with multicast support), latency

Distributed transactions

- 2PC state diagram:



Distributed transactions

- Non-blocking protocols (additional property):
  - (5) correct participants will decide sooner or later
- In a blocking protocol, correct participants may have to wait for the recovery of failed ones
  - meanwhile, data locks cannot be released
- 2PC eta D2PC protocols are blocking!
  - if the coordinator (2PC) or any participant (D2PC) fails in a critical instant
- Three Phase Commit (3PC): non-blocking

Two-Phase Commit (1)

actions by coordinator:

```
write START_2PC to local log;
multicast VOTE_REQUEST to all participants;
while not all votes have been collected {
    wait for any incoming vote;
    if timeout {
        write GLOBAL_ABORT to local log;
        multicast GLOBAL_ABORT to all participants;
        exit;
    }
    record vote;
}
if all participants sent VOTE_COMMIT and coordinator votes COMMIT {
    write GLOBAL_COMMIT to local log;
    multicast GLOBAL_COMMIT to all participants;
} else {
    write GLOBAL_ABORT to local log;
    multicast GLOBAL_ABORT to all participants;
}
```

Two-Phase Commit (2)

actions by participant:

```
write INIT to local log;
wait for VOTE_REQUEST from coordinator;
if timeout {
    write VOTE_ABORT to local log;
    exit;
}
if participant votes COMMIT {
    write VOTE_COMMIT to local log;
    send VOTE_COMMIT to coordinator;
    wait for DECISION from coordinator;
    if timeout {
        multicast DECISION_REQUEST to other participants;
        wait until DECISION is received;   /* remain blocked */
        write DECISION to local log;
    }
    if DECISION == GLOBAL_COMMIT
        write GLOBAL_COMMIT to local log;
    else if DECISION == GLOBAL_ABORT
        write GLOBAL_ABORT to local log;
} else {
    write VOTE_ABORT to local log;
    send VOTE_ABORT to coordinator;
}
```

Two-Phase Commit (3)

actions for handling decision requests: /* executed by separate thread */

```
while true {
    wait until any incoming DECISION_REQUEST is received;
    read most recently recorded STATE from the local log;
    if STATE == GLOBAL_COMMIT
        send GLOBAL_COMMIT to requesting participant;
    else if STATE == INIT or STATE == GLOBAL_ABORT
        send GLOBAL_ABORT to requesting participant;
    else
        skip;   /* participant remains blocked */
```

The Consensus problem

- In the Consensus problem, each process proposes a value, and processes must decide on a value among the proposed ones
- Applications:
  - total order broadcast: “which is the next message to deliver?”
  - group membership management: “which is next view?”
  - election: “which is the new coordinator/leader?”
  - atomic commitment: “shall we *commit* or *abort*?”
- Primitives:
  - *propose(v)*: the process proposes *v*
  - *decide(v)*: the process decides *v*

The Consensus problem

- Problem specification:
  - Termination: every correct process eventually decides some value
  - Uniform integrity: every process decides at most once
  - Uniform validity: if a process decides *v*, then *v* was proposed by some process
  - Agreement: no two correct processes decide differently
    - Uniform agreement: no two processes, *whether correct or faulty*, decide differently
- Validity captures the relation between the decision value and the proposed values

The Consensus problem

- FLP impossibility: in asynchronous systems subject to failures, the Consensus problem cannot be solved *deterministically*
  - even if communication channels are reliable, and at most one process can fail by crashing
  - intuitively, in asynchronous systems it is not possible to detect process failures reliably
- Proposals for circumventing FLP:
  - non-deterministic solutions (probabilistic)
  - consider a partially synchronous system model
  - unreliable failure detectors